

A unifying survey on weighted logics and weighted automata

Core weighted logic: minimal and versatile specification of quantitative properties

Paul Gastin¹ · Benjamin Monmege²

© Springer-Verlag Berlin Heidelberg 2015

Abstract Logical formalisms equivalent to weighted automata have been the topic of numerous research papers in the recent years. It started with the seminal result by Droste and Gastin on weighted logics over semirings for words. It has been extended in two dimensions by many authors. First, the weight domain has been extended to valuation monoids, valuation structures, etc. to capture more quantitative properties. Along another dimension, different structures such as ranked or unranked trees, nested words, Mazurkiewicz traces, etc. have been considered. The long and involved proofs of equivalences in all these papers are implicitly based on the same core arguments. This article provides a meta-theorem which unifies these different approaches. Towards this, we first revisit weighted automata by defining a new semantics for them in two phases—an abstract semantics based on multisets of weight structures (independent of particular weight domains) followed by a concrete semantics. Then, we introduce a core weighted logic with a minimal number of features and a simplified syntax, and lift the new semantics to this logic. We show at the level of the abstract seman-

tics that weighted automata and core weighted logic have the same expressive power. Finally, we show how previous results can be recovered from our result by logical reasoning. In this paper, we prove the meta-theorem for words, ranked and unranked trees, showing the robustness of our approach.

1 Introduction

Weighted automata are a well-studied formalism modelling quantitative behaviours. Introduced by Schützenberger (1961), they have been applied in many areas such as image compression (Albert and Kari 2009), natural language processing (Knight and May 2009), verification and synthesis of programs (Černý et al. 2011), etc. In the last years, high-level specification formalisms of quantitative properties have received increasing interest. Among other successes, the connection between monadic second-order logic (MSO) and finite automata established by Büchi, Elgot and Trakhtenbrot (Büchi 1960; Elgot 1961; Trakhtenbrot 1961), has been extended to the weighted setting.

There have been many attempts to find a suitable extension of MSO to describe quantitative properties which captures the expressive power of weighted automata. The considered variants differ with respect to the structures (words, ranked or unranked trees, nested words, etc) and the weight domains (semirings, valuation monoids, valuation structures, multi-operator monoids, etc). This article aims at revisiting the link between weighted logics and weighted automata in a uniform manner with regards of these two dimensions.

Our main contribution is to consider a new fragment of weighted logics containing a minimal set of features. In order to simplify the uniformity with respect to the structures, we syntactically separate a Boolean fragment from the weighted part: only the syntax of Boolean formulae depends on the

Communicated by M. Droste, Z. Esik and K. Larsen.

Part of the research leading to these results was achieved when the second author was at Université libre de Bruxelles (Belgium), and has received funding from the European Union Seventh Framework Programme (FP7/2007–2013) under Grant Agreement n601148 (CASSTING).

✉ Benjamin Monmege
benjamin.monmege@lif.univ-mrs.fr

Paul Gastin
paul.gastin@lsv.ens-cachan.fr

¹ LSV ENS Cachan, CNRS Université Paris-Saclay, 94235 Cachan, France

² Aix-Marseille Université, LIF, CNRS, Marseille, France

structures considered. Then, we clearly separate a small fragment able to define step functions—that we call step formulæ—from the more general weighted logic. Because of the minimal set of features that it displays, we call our logic core weighted monadic second-order logic. This separation into three distinct layers, more or less clear in previous works, is designed both to clarify the subsequent study of the expressive power, and to simplify the use of the weighted logic.

Towards defining the semantics of this new logic, we first revisit weighted automata by defining an alternative semantics, then lifting it to formulæ. This is done in two phases. First, an abstract semantics associates with a structure a multiset of weight labelled structures. E.g. in the case of words, a weighted automaton/formula will map every word to a multiset of weight words. In the setting of trees, every tree is associated with a multiset of weight trees (of the same shape as the original tree). This abstract semantics is fully uninterpreted and, hence, does not depend on any algebraic structure over the set of weights considered. This semantics is in the spirit of a transducer. It has already been used in similar contexts: by Fülöp (2012) with an operator $H(\omega)$ which relabels trees with operations taken from a multi-operator monoid, by Perevoshchikov (2015) with a weight assignment logic over infinite words, by Droste and Perevoshchikov (2014); Perevoshchikov (2015) with Nivat theorems for weighted automata over various structures. In a second phase, a concrete semantics is given, by means of an aggregation operator taking the abstract semantics and aggregating every multiset of weight structures to a single value (in a possibly different weight domain). For instance, the usual semantics of weighted automata over semirings can be recovered by mapping every weight word to the product of its weights, and merging the multiset with the addition of the semiring.

Separating the semantics in two successive phases, both for weighted automata and logics, allows us to revisit the original proof of expressive equivalence of Droste and Gastin (2007) in the abstract semantics. This result has been extended to various weight domains and/or structures (see below). The proof of equivalence in all these works is based on the same core argument which relates runs of automata with the evaluation of formulæ. Inspired by the above similarities, our choice of the abstract multiset semantics manifests this core argument. Because the abstract semantics is fully uninterpreted, no additional hypotheses on the weight domain are required to prove the equivalence. We then apply the aggregation operator to obtain a concrete equivalence between weighted automata and our core weighted logic.

Our last contribution is to show, by means of purely logical reasoning, that our new fragment of core weighted logic is expressively equivalent to the logics proposed in the previous works. Over finite words, this allows us to recover the results over semirings (Droste and Gastin 2007), (product)

valuation monoids (Droste and Meinecke 2012) and (product) valuation structures (Droste and Perevoshchikov 2013). Valuation monoids replace the product operation of the semiring by a lenient valuation operation, making possible to consider discounted sums, average or more evolved combination of sequences of weights. Valuation structures finally also replace the sum by a more general evaluation operator, for instance ratios of several weights computed simultaneously. As an example, it is then possible to compute the ratio between rewards and costs, or the efficiency of use of a primary resource under some upper bound constraint on a secondary resource. Our unifying proof gives new insights on the additional hypotheses (commutativity, distributivity, etc) over the weight domains used in these works.

After studying in full details the case of finite words, we illustrate the uniformity of the method with respect to structures, by considering ranked and unranked trees. Once again, our study revisits existing works over semirings (Droste 2006; Droste and Vogler 2011), (product) valuation monoids (Droste et al. 2011) and also multi-operator monoids (Fülöp 2012). The syntax of the logic in the case of multi-operator monoids is different from the other logics. The proof techniques used to show equivalence of the two formalisms are nevertheless very close to the original ones for semirings.

2 Preliminaries

In this section, we recall the basic notions of semirings and multisets that we will need in the following.

A semiring is a tuple $(S, +, \times, 0, 1)$ where $+$ and $\times$ are two binary operations, and 0 and 1 are two elements of S , verifying the following:

- $(S, +, 0)$ is a commutative monoid;
- $(S, \times, 1)$ is a monoid;
- $\times$ distributes over $+$;
- 0 is a zero, i.e. $s \times 0 = 0 \times s = 0$ for all $s \in S$.

Classical examples of semirings are given by $(\mathbb{Z}, +, \times, 0, 1)$, $(\mathbb{N} \cup \{-\infty\}, \max, +, -\infty, 0)$, $(\mathbb{R} \cup \{+\infty\}, \min, +, +\infty, 0)$, $(\mathfrak{P}(\Sigma^*), \cup, \cdot, \emptyset, \{\varepsilon\})$ where Σ is a finite alphabet and $\cdot$ denotes the concatenation. More examples can be found in, e.g. (Droste et al. 2009; Kuich et al. 1985).

Multisets generalise sets by allowing several occurrences of the same element in a multiset. For instance, $A = \{a, a, b, b, b, c\}$ is a finite multiset consisting of two occurrences of a , three occurrences of b and one occurrence of c . Formally, a multiset A over a set X is a (multiplicity) function $A: X \rightarrow \mathbb{N}$ which gives for each $x \in X$ the number $A(x)$ of occurrences of x in A . The size of a multiset is denoted by $|A| = \sum_{x \in X} A(x)$, and we denote by $x \in A$ the fact that $A(x) \neq 0$.

The disjoint union of multisets, defined by the pointwise sum $(A \uplus B)(x) = A(x) + B(x)$ for all $x \in X$, is a commutative and associative operation on multisets with the empty multiset, denoted by $\emptyset$, as neutral element. A multiset A is finite if its support $\text{supp}(A) = \{x \in X \mid A(x) \neq 0\}$ is finite. We denote by $\mathbb{N}\langle X \rangle$ the set of finite multisets over X . Clearly, $(\mathbb{N}\langle X \rangle, \uplus, \emptyset)$ is a commutative monoid.

Every operation defined on the set X can be lifted to multisets over X . For instance, if $\diamond$ is a binary operation over X , then it is lifted as a binary operation over $\mathbb{N}\langle X \rangle$ by $A \diamond B = \{\{a \diamond b \mid a \in A, b \in B\}\}$: formally, for all $x \in X$, we let

$$(A \diamond B)(x) = \sum_{\substack{(x_1, x_2) \in X^2 \\ x = x_1 \diamond x_2}} A(x_1) \times B(x_2).$$

For instance, if $X = \mathbb{N}$ and $\diamond = \times$ is the usual product, we get $\{\{1, 2, 2\} \times \{3, 6\}\} = \{\{3, 6, 6, 6, 12, 12\}\}$.

Similarly, a unary operation $f: X \rightarrow Y$ is lifted as a unary operation from $\mathbb{N}\langle X \rangle$ to $\mathbb{N}\langle Y \rangle$ by $f(A) = \{\{f(a) \mid a \in A\}\}$: formally, for all $y \in Y$, we let

$$(f(A))(y) = \sum_{\substack{x \in X \\ f(x) = y}} A(x).$$

For instance, if $X = \mathbb{Z}$, $Y = \mathbb{N}$, $f(x) = x^2$ and $A = \{-2, -1, -1, 0, 1, 3\}$ then $f(A) = \{0, 1, 1, 1, 4, 9\}$.

When $(Y, +, 0)$ is a commutative monoid and $B \in \mathbb{N}\langle Y \rangle$ then we let

$$\sum B = \sum_{y \in Y} (\underbrace{y + \dots + y}_{B(y) \text{ times}})$$

be the sum of the elements in B (with multiplicities), where $\sum$ denotes the addition in the monoid. For instance, with notations of the previous example, we have $\sum f(A) = 16$. Note that $\sum \emptyset = 0$.

Multisets over a monoid form a semiring. More precisely, let $(M, \times, \mathbf{1})$ be a monoid. Then, lifting $\times$ to $\mathbb{N}\langle M \rangle$ yields a monoid with $\{\mathbf{1}\}$ as neutral element. Moreover, $\times$ distributes over $\uplus$ and $\emptyset$ is a zero for $\times$. Therefore, $(\mathbb{N}\langle M \rangle, \uplus, \times, \emptyset, \{\mathbf{1}\})$ is a semiring.

In this paper, we will mainly use the free monoid $(R^*, \cdot, \varepsilon)$ over a finite set R of weights, and the induced semiring $(\mathbb{N}\langle R^* \rangle, \uplus, \cdot, \emptyset, \{\varepsilon\})$.

3 Weighted automata

Weighted automata are a well-studied model of computation of formal power series introduced by [Schützenberger](#)

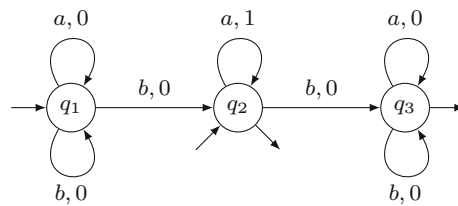


Fig. 1 A $\{0, 1\}$ -weighted automaton over $\{a, b\}$

(1961). Originally introduced over integers, they have afterwards been extended to more general weighted domains. We recall here the syntax of weighted automata, and then give the semantics in various weight structures, before revisiting this semantics with our new abstract semantics.

3.1 Syntax of weighted automata

Formally, a weighted automaton is simply a classical finite-state automaton over a finite alphabet Σ , where every transition is equipped with a weight taken from a set R . Precisely, an R -weighted automaton over Σ is a tuple $\mathcal{A} = (Q, \Delta, \text{wgt}, I, F)$ with Q a non-empty finite set of states, $\Delta \subseteq Q \times \Sigma \times Q$ the set of transitions, $\text{wgt}: \Delta \rightarrow R$ associating a weight to every transition and $I, F \subseteq Q$ are, respectively, initial and final states.

An example of a $\{0, 1\}$ -weighted automaton over alphabet $\Sigma = \{a, b\}$ is given in Fig. 1. It has three states q_1, q_2 and q_3 , the first two being initial and the last two being final. The weights of transitions are depicted on the right of their labels: the only transition of weight different from 0 has weight 1 and is the loop on state q_2 .

A run of $\mathcal{A}$ over a word $w = a_1 \dots a_n \in \Sigma^+$ is a sequence of transitions $\rho = \delta_1 \dots \delta_n \in \Delta^+$ with $\delta_i = (q_i, a_i, q'_i)$ for all $1 \leq i \leq n$ such that $q'_i = q_{i+1}$ for all $1 \leq i \leq n-1$. The run ρ is accepting if it starts in an initial state ($q_1 \in I$), and ends in a final state ($q'_n \in F$). For the weighted automaton of Fig. 1, the sequence $(q_1, a, q_1), (q_1, b, q_2), (q_2, a, q_2), (q_2, a, q_2), (q_2, b, q_3)$ is an accepting run over the word $abaab$.

3.2 Semantics over semirings

When the set R of weights is a subset of a semiring $(S, +, \times, 0, 1)$, weighted automata are called weighted automata over semirings. The usual quantitative semantics on a word $w \in \Sigma^+$ is obtained as follows. First, we compute the value of a run $\rho = \delta_1 \dots \delta_n$ on w using the multiplication in the semiring: $\text{Val}(\rho) = \text{wgt}(\delta_1) \times \dots \times \text{wgt}(\delta_n)$. Then, the semantics of $\mathcal{A}$ on w is obtained by taking the sum in the semiring of the values of accepting runs:

$$[\![\mathcal{A}]\!](w) = \sum_{\rho} \text{Val}(\rho) \quad (1)$$

where ρ ranges over all accepting runs over w .

For instance, consider the $\{0, 1\}$ -weighted automaton of Fig. 1 over the semiring $(\mathbb{N} \cup \{-\infty\}, \max, +, -\infty, 0)$. Each run uses the loop over the state q_2 to count the length of a block of a 's in the word. Hence, we can check that the semantics of the automaton maps every word w to the maximal length of a block of a 's in w . For instance, $\llbracket \mathcal{A} \rrbracket(abaaba) = 2$.

3.3 Semantics over valuation monoids

Several extensions of weighted automata over semirings have been proposed. We start here with valuation monoids (Droste and Meinecke 2012) and we will discuss valuation structures (Droste and Perevoshchikov 2013) in the next subsection.

The motivation comes from the fact that, for certain applications, the value of a run cannot be computed with a product in a semiring. For instance, the value of a run may be a discounted sum of the weights of transitions, or the average of the weights. An idea is to replace the product of the semiring by a valuation function—without the conditions that the product of a semiring must verify—which computes the value of a sequence of weights.

A valuation monoid is a tuple $(S, +, 0, \text{Val})$ where $(S, +, 0)$ is a commutative monoid and $\text{Val}: S^* \rightarrow S$ is a function, called valuation operator that maps each finite sequence of elements of S to an element of S . In the original work of Droste and Meinecke (2012), the valuation operator is only defined on non-empty sequences from S^+ but we may extend it by setting $\text{Val}(\varepsilon)$ to an arbitrary value. Also, in the definition of Droste and Meinecke (2012), Val must satisfy some additional properties, such as $\text{Val}(s) = s$ for all $s \in S$, and for $s_1 \dots s_n \in S^+$ we have $\text{Val}(s_1 \dots s_n) = 0$ whenever $s_i = 0$ for some i . However, these additional conditions are not necessary to define the semantics of weighted automata and to obtain the equivalence between weighted automata and our weighted logic that will be presented in Sect. 4.

For instance, we may use as valuation a discounted sum $\text{Val} = \text{Disc}_\lambda$ defined for $\lambda \in (0, 1)$ and a sequence of real numbers by $\text{Disc}_\lambda(s_1 \dots s_n) = s_1 + \lambda s_2 + \dots + \lambda^{n-1} s_n$. It is also possible to consider as valuation the average $\text{Val} = \text{Avg}$ of a sequence of real numbers, given by $\text{Avg}(s_1 \dots s_n) = \frac{1}{n}(s_1 + \dots + s_n)$. In those two cases, $+$ denotes the usual addition over real numbers, and we may consider the monoid operation to be max or min for instance, allowing us to compute optimal discounted or average costs. We may also use the sum $+$ which is not allowed by Droste and Meinecke (2012) since it would not fulfil the additional conditions given above.

We now give the semantics of R -weighted automata with $R \subseteq S$, and $(S, +, 0, \text{Val})$ a valuation monoid: in the following, such automata are called weighted automata over valuation monoids. Let $\mathcal{A}$ be an R -weighted automaton and $w = a_1 \dots a_n \in \Sigma^+$. As expected, the value of a run

$\rho = \delta_1 \dots \delta_n$ on w is computed with the valuation operator $\text{Val}(\rho) = \text{Val}(\text{wgt}(\delta_1) \dots \text{wgt}(\delta_n))$. Then, the semantics of $\mathcal{A}$ over w is defined by (1) as before.

As an example, consider again the $\{0, 1\}$ -weighted automaton of Fig. 1 and the valuation monoid $(\mathbb{R} \cup \{-\infty\}, \max, \text{Disc}_\lambda, -\infty)$. Then, the semantics of the automaton on a word $w \in \Sigma^+$ is the maximal score of the blocks of a 's. The score of a block is computed as $\lambda^{i-1} \frac{\lambda^\ell - 1}{\lambda - 1}$, where i is the position of the first letter of the block and ℓ is its length.

As another example, consider the valuation monoid $(\mathbb{N} \cup \{\infty\}, \min, \text{Sq}, \infty)$ where the valuation¹ Sq maps a sequence $s_1 \dots s_n$ to the square of the number of s_i 's which are equal to 1. Then, the semantics of the weighted automaton of Fig. 1 is the minimum of the squares of the lengths of a -blocks.

3.4 Semantics over valuation structures

In order to get even more flexibility in the computation of the semantics, Droste and Perevoshchikov (2013) proposed to replace the sum of the run values which are used in (1) by a more general evaluator function F . In addition, Droste and Perevoshchikov (2013) allows the set of weights R used in the automaton and for the value of runs to be different from the final set of weights S computed by the evaluator function.

Formally, a valuation structure² is a tuple (U, Val, S, F) where U and S are two sets, $\text{Val}: U^* \rightarrow U$ is a valuation operator, and $F: \mathbb{N}\langle U \rangle \rightarrow S$ is an evaluator function mapping a finite multiset of weights in U to a single weight in S .

Given an R -weighted automaton $\mathcal{A}$ with $R \subseteq U$ (that we call weighted automata over a valuation structure), we compute the value of a run as in the case of valuation monoids: $\text{Val}(\rho) = \text{Val}(\text{wgt}(\delta_1) \dots \text{wgt}(\delta_n))$ when $\rho = \delta_1 \dots \delta_n$. Then, the semantics of $\mathcal{A}$ over a word $w \in \Sigma^+$ is defined in two steps:

$$\llbracket \mathcal{A} \rrbracket(w) = F(\{\{\text{Val}(\rho) \mid \rho \text{ accepting run over } w\}\}).$$

First, Val transforms the set of accepting runs over w in a multiset of weights, which is then transformed in the final semantics with the evaluator function.

For instance, we may choose $U = \mathbb{Z} \times \mathbb{N}$ with the valuation

$$\text{Val}((x_1, y_1), \dots, (x_n, y_n)) = \left(\sum_{i=1}^n x_i, \sum_{i=1}^n y_i \right).$$

Then, choosing $S = \mathbb{Q} \cup \{+\infty\}$ in the valuation structure, we may compute the average of the ratios between rewards

¹ This valuation does not satisfy $\text{Sq}(s) = s$.

² We do not require that $\text{Val}(r) = r$ for all $r \in U$ as in Droste and Perevoshchikov (2013).

and non-negative costs with the evaluator function defined by $F(\emptyset) = 0$ and

$$F(A) = \frac{1}{|A|} \sum_{(x,y) \in A} \frac{x}{y}$$

for $A \in \mathbb{N}\langle U \rangle \setminus \{\emptyset\}$, with $\frac{x}{0} = +\infty$ by convention. We may change the evaluator function to

$$F(A) = \max_{(x,y) \in A} \frac{x}{y}$$

to compute the maximal ratio between rewards and costs (with $-\infty$ added to S and $\max(\emptyset) = -\infty$). We may further change the evaluator function to

$$F_p(A) = \max\{x \mid (x, y) \in A \wedge y \leq p\}$$

with p any positive number, to compute the best reward under the given cost upper bound.

As a last example, consider the valuation structure $(\mathbb{N}, \text{Val}, \mathbb{Q}, F)$, where the valuation Val sums the sequence of weights, and the evaluator function $F = \text{Avg}$ computes the average of the multiset as $\text{Avg}(\emptyset) = 0$ and

$$\text{Avg}(A) = \frac{1}{|A|} \sum A$$

for $A \neq \emptyset$. Then, the $\{0, 1\}$ -weighted automaton of Fig. 1 computes the average length of the a -blocks of the word.

3.5 Abstract semantics

After recalling some existing semantics of weighted automata, we now present a unifying framework, mostly based on weighted automata over semirings that will still be able to recover as special cases all previous semantics.

The main idea is to split the semantics into two phases, as already done for valuation structures, but the separation takes place somewhere else in order to obtain an intermediary structure with even more information. Instead of multisets of values in R , we will use multisets of sequences in R^* . Interestingly, since $(R^*, \cdot, \varepsilon)$ is a monoid, we have shown in Sect. 2 that $(\mathbb{N}\langle R^* \rangle, \sqcup, \cdot, \emptyset, \{\{\varepsilon\}\})$ is a semiring. This remark will be useful in the following. As usual, we will not indicate the operation $\cdot$ in expressions over $\mathbb{N}\langle R^* \rangle$.

The abstract semantics of an R -weighted automaton $\mathcal{A}$ is simply its semantics in the semiring $\mathbb{N}\langle R^* \rangle$ when we identify the weights $r \in R$ occurring in $\mathcal{A}$ with the singleton multisets $\{r\}$. Computing in the semiring $\mathbb{N}\langle R^* \rangle$, the value $\text{Val}(\rho)$ of a run $\rho = \delta_1 \dots \delta_n$ over some word $w \in \Sigma^+$ is

$$\{\text{wgt}(\delta_1)\} \dots \{\text{wgt}(\delta_n)\} = \{\text{wgt}(\delta_1) \dots \text{wgt}(\delta_n)\}.$$

The abstract semantics, denoted by $\llbracket \mathcal{A} \rrbracket(w)$, is obtained by summing (i.e. taking the disjoint union) over all accepting runs over w :

$$\llbracket \mathcal{A} \rrbracket(w) = \bigsqcup_{\rho = \delta_1 \dots \delta_n} \{\text{wgt}(\delta_1) \dots \text{wgt}(\delta_n)\}.$$

Notice that $\llbracket \mathcal{A} \rrbracket(w)$ is indeed a finite multiset, since $\mathcal{A}$ admits only finitely many accepting runs over w .

Example 1 Consider the $\{0, 1\}$ -weighted automaton of Fig. 1. Its abstract semantics over the word $abaab$ is $\llbracket \mathcal{A} \rrbracket(abaab) = \{\{00000, 00110, 10000\}\}$.

Remark 2 Our abstract semantics proposes another level of abstraction compared to the one obtained with Nivat theorems for weighted automata, e.g. in [Droste and Perevoshchikov \(2014\)](#); [Perevoshchikov \(2015\)](#). In these Nivat-like characterisations, the behaviour of a weighted automaton is described by its set of runs, whereas we use the less concrete description by its multiset of weight structures (here, sequences of weights). Our more abstract semantics is sufficient for our later discussion. Also, it allows us to keep an underlying semiring that permits to reuse initial results for weighted automata over semirings.

From this abstract semantics, we compute the concrete semantics by aggregating the multiset of weight sequences into a single weight, possibly lying in a different weight domain S . We do so with an aggregation operator $\text{aggr}: \mathbb{N}\langle R^* \rangle \rightarrow S$. We obtain the concrete semantics $\llbracket \mathcal{A} \rrbracket(w) = \text{aggr}(\llbracket \mathcal{A} \rrbracket(w))$. The remaining of this section is devoted to some examples, covering as special cases the various semantics shown in Sect. 2.

Example 3 If R is a subset of a semiring $(S, +, \times, 0, 1)$, we recover the usual semantics by considering the aggregation operator

$$\text{aggr}_{\text{sr}}(A) = \sum \prod(A),$$

where $\prod: S^* \rightarrow S$ maps every sequence $r_1 \dots r_n \in S^*$ to the product $r_1 \times \dots \times r_n$, and is lifted over finite multisets as explained in the preliminaries.

Example 4 The previous example can be generalised to the case where R is a subset of a valuation monoid $(S, +, 0, \text{Val})$. Using the lift of $\text{Val}: S^* \rightarrow S$ to finite multisets, the aggregation operation is now

$$\text{aggr}_{\text{vm}}(A) = \sum \text{Val}(A).$$

Example 5 We can also recover the semantics in a valuation structure (U, Val, S, F) by considering the aggregation defined by

$$\text{aggr}_{\text{vs}}(A) = F(\text{Val}(A)).$$

Notice that our framework is a priori more powerful than valuation structures. Indeed, in our context, we do not decouple the valuation and the evaluator operations. Instead, we keep a richer information, namely the multiset of weight sequences, allowing more powerful aggregations.

For instance, imagine that, on the weighted automaton of Fig. 1, we want to compute the discounted sum of the length of blocks of a 's. E.g. if $\lambda \in (0, 1)$ is the discount factor, we would like to associate to the word $abaababaaa$ the weight $1 + \lambda \cdot 2 + \lambda^2 \cdot 1 + \lambda^3 \cdot 3$. For that purpose, consider R to be $\{0, 1\}$, S to be $\mathbb{R}$, and the aggregation aggr to be defined on a multiset $A \in \mathbb{N}\langle R^* \rangle$ by

$$\text{aggr}(A) = \sum_{j=1}^k \lambda^{j-1} n_j$$

if $A = \{0^{m_1} 1^{n_1} 0^{p_1}, 0^{m_2} 1^{n_2} 0^{p_2}, \dots, 0^{m_k} 1^{n_k} 0^{p_k}\}$ with $m_1 < m_2 < \dots < m_k$, and $\text{aggr}(A) = 0$ otherwise. This aggregation cannot be obtained directly in the setting of valuation structures, since the value of a run should now take into account the discount factor, based on the number of blocks of a 's previously seen: however, this number cannot be obtained by directly looking at the unique current run.

4 Core weighted monadic second-order logic

We now turn to the description of a new weighted logic that will be equivalent to weighted automata. Most existing works start with the definition of a very general logic, and then introduce restrictions to match the expressive power of weighted automata. We take the opposite approach by defining a very basic weighted logic, yet powerful enough to be expressively equivalent to weighted automata. Our logic has three layers: the Boolean fragment which is the classical MSO logic over words, a step weighted fragment (**step-wMSO**) defining step functions (i.e. piecewise constant functions with a finite number of pieces), and the core weighted logic (**core-wMSO**) which has the full expressive power of weighted automata. We will show in Sect. 5 that **core-wMSO** is a fragment of the (full) weighted MSO logic (**wMSO**) defined in [Droste and Gastin \(2007\)](#). Considering a Boolean fragment inside a weighted logic was originally done in [Bollog and Gastin \(2009\)](#) and followed in many articles, see, e.g. ([Droste and Meinecke 2012](#); [Fülöp 2012](#)).

4.1 Syntax and semantics

We fix a finite alphabet Σ and we consider a set $\mathcal{V}$ of first-order $(x, y, \dots)$ and second-order $(X, Y, \dots)$ variables. The syntax of **MSO**(Σ) is given in Table 1 (**MSO**). We may freely use classical macros such as $\perp = \neg \top$, $\varphi \vee \varphi' = \neg(\neg\varphi \wedge \neg\varphi')$ or $\exists x \varphi = \neg \forall x \neg \varphi$.

We do not recall the semantics of an **MSO** formula ϕ which can be defined inductively as usual. We denote by $\text{free}(\varphi)$ the set of free variables in φ . Given a word $w \in \Sigma^+$ and a valuation σ from a set of variables $V \supseteq \text{free}(\varphi)$ to the (sets of) positions of w , we write $w, \sigma \models \phi$ when (w, σ) satisfies ϕ .

We will use the classical encoding of a pair (w, σ) consisting of a word $w \in \Sigma^+$ and a valuation σ on V as a word $\bar{w}$ over the extended alphabet $\Sigma_V = \Sigma \times \{0, 1\}^V$. A word $\bar{w}$ in Σ_V^+ is said to be valid if for every first-order variable $x \in V$, the projection of $\bar{w}$ on the component indexed by x belongs to $0^* 1 0^*$. In the following, we identify a valid word $\bar{w}$ with its encoded pair (w, σ) .

The syntax for **step-wMSO**(Σ, R) formulae is given in Table 1 **step-wMSO**. It uses weights from a set R and is based on the if-then-else operator $\varphi ? \Psi_1 : \Psi_2$ where the condition is an **MSO** formula φ .

The semantics of such formulae is given in terms of finite multisets of weights. More precisely, for a set V of variables that contain the set $\text{free}(\Psi)$ of free variables of Ψ , we let $\|\cdot\|_V : \Sigma_V^+ \rightarrow \mathbb{N}\langle R \rangle$ be defined by $\|\Psi\|(\bar{w}) = \emptyset$ if $\bar{w}$ is not valid, and if $\bar{w}$ is the valid encoding of (w, σ) then the semantics is given in Table 2. Notice that $\|\Psi\|(\bar{w})$ is either the empty multiset (if $\bar{w}$ is not a valid encoding) or is a singleton multiset. We simply write $\|\Psi\|$ for $\|\Psi\|_{\text{free}(\Psi)}$.

Finally, the syntax of the **core-wMSO**(Σ, R) logic is given in Table 1 **core-wMSO**. It uses a special constant $\mathbf{0}$. In addition to the if-then-else operator, there are three sum operators whose semantics will be defined using the disjoint union (sum) of multisets, and a product which lifts the weights defined by **step-wMSO** formulae to sequences of weights.

Remark 6 The idea of splitting the syntax into three layers has also been used by Perevoshchikov to define the weighted assignment logic over infinite words Chapter 6, ([Perevoshchikov 2015](#)). There, the fragment uWAL is related to our **step-wMSO** and the full logic WAL allows one to apply sums, as in our **core-wMSO** logic.

Formally, the semantics of **core-wMSO** is again defined by induction on the formula. For a set V of variables that contain the set $\text{free}(\Phi)$ of free variables of Φ , we let $\|\cdot\|_V : \Sigma_V^+ \rightarrow \mathbb{N}\langle R^* \rangle$ be defined by $\|\Phi\|(\bar{w}) = \emptyset$ if $\bar{w}$ is not valid, and if $\bar{w}$ is the valid encoding of (w, σ) then the semantics is given in Table 3. We simply write $\|\Phi\|$ for $\|\Phi\|_{\text{free}(\Phi)}$.

Table 1 Syntax of the core weighted logic
core-wMSO(Σ, R)

$\phi ::= \top \mid P_a(x) \mid x \leq y \mid x \in X \mid \neg\phi \mid \phi \wedge \phi \mid \forall x\phi \mid \forall X\phi$	(MSO)
$\Psi ::= r \mid \phi ? \Psi : \Psi$	(step-wMSO)
$\Phi ::= \mathbf{0} \mid \phi ? \Phi : \Phi \mid \Phi + \Phi \mid \sum_x \Phi \mid \sum_X \Phi \mid \prod_x \Psi$	(core-wMSO)
with $a \in \Sigma, r \in R, x, y$ first-order variables and X a second-order variable	

Table 2 Semantics of (step – wMSO)

$\llbracket r \rrbracket_V(w, \sigma) = \{\{r\}\}$
$\llbracket \phi ? \Psi_1 : \Psi_2 \rrbracket_V(w, \sigma) = \begin{cases} \llbracket \Psi_1 \rrbracket_V(w, \sigma) & \text{if } w, \sigma \models \phi \\ \llbracket \Psi_2 \rrbracket_V(w, \sigma) & \text{otherwise} \end{cases}$

Table 3 Semantics of (core-wMSO)

$\llbracket \mathbf{0} \rrbracket_V(w, \sigma) = \emptyset$
$\llbracket \varphi ? \Phi_1 : \Phi_2 \rrbracket_V(w, \sigma) = \begin{cases} \llbracket \Phi_1 \rrbracket_V(w, \sigma) & \text{if } w, \sigma \models \varphi \\ \llbracket \Phi_2 \rrbracket_V(w, \sigma) & \text{otherwise} \end{cases}$
$\llbracket \Phi_1 + \Phi_2 \rrbracket_V(w, \sigma) = \llbracket \Phi_1 \rrbracket_V(w, \sigma) \uplus \llbracket \Phi_2 \rrbracket_V(w, \sigma)$
$\llbracket \sum_x \Phi \rrbracket_V(w, \sigma) = \biguplus_{i \in \text{pos}(w)} \llbracket \Phi \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i])$
$\llbracket \sum_X \Phi \rrbracket_V(w, \sigma) = \biguplus_{I \subseteq \text{pos}(w)} \llbracket \Phi \rrbracket_{V \cup \{X\}}(w, \sigma[X \mapsto I])$
$\llbracket \prod_x \Psi \rrbracket_V(w, \sigma) = \prod_{i=1}^{ w } \llbracket \Psi \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i])$
where $\sigma[x \mapsto i]$ denotes the valuation obtained from σ by mapping x to i and keeping the other assignments unchanged (and similarly for second-order variables). Note that $\prod_{i=1}^{ w }$ is the product in the semiring $\mathbb{N}\langle R^* \rangle$

Example 7 We give a core-wMSO($\{a, b\}, \{0, 1\}$) formula equivalent with the $\{0, 1\}$ -weighted automaton of Fig. 1:

$$\Phi = \sum_X \sum_y \text{block}(X, y) ? \prod_x (x \in X ? 1 : 0) : \mathbf{0}$$

where the MSO formula $\text{block}(X, y)$ defined by

$$\begin{aligned} & \left[X = \emptyset \wedge (\text{first}(y) \vee \text{last}(y)) \wedge P_b(y) \right] \\ & \vee \left[(\text{last}(y) \vee P_b(y+1)) \wedge \right. \\ & \quad \left. \exists x (x \leq y \wedge (\text{first}(x) \vee P_b(x-1)) \wedge \right. \\ & \quad \left. \forall z ((x \leq z \leq y \Leftrightarrow z \in X) \wedge (z \in X \Rightarrow P_a(z))) \right) \end{aligned}$$

states that X is a (possibly empty) maximal block of consecutive a 's ending in y . Variable y enables us to distinguish two reasons for X to be empty: either because the word starts with b or ends with b . Notice the use of the constant $0 \in R$ in the step-wMSO formula $x \in X ? 1 : 0$, not to be confused with the atomic formula $\mathbf{0}$ of core-wMSO. Semantically, we have

$$\llbracket \Phi \rrbracket(baabab) = \{\{000000, 011000, 000010, 000000\}\}.$$

We say that two core-wMSO formulæ Φ and Φ' are equivalent, written $\Phi \equiv \Phi'$, if they have the same semantics: $\text{free}(\Phi) = \text{free}(\Phi')$ and $\llbracket \Phi \rrbracket(w, \sigma) = \llbracket \Phi' \rrbracket(w, \sigma)$ for all $(w, \sigma) \in \Sigma_{\text{free}(\Phi)}^+$. The equivalence of step-wMSO formulæ is defined similarly.

As in Sect. 3.5, we apply an aggregation function $\text{aggr}: \mathbb{N}\langle R^* \rangle \rightarrow S$ transforming a finite multiset of weight sequences from R into a weight in a (possibly new) set S . We obtain the concrete semantics $\llbracket \Phi \rrbracket(\bar{w}) = \text{aggr}(\llbracket \Phi \rrbracket(\bar{w}))$.

Remark 8 Notice that some operators of step-wMSO and core-wMSO can be removed without affecting the expressive power of these logics. For instance, we will show in Lemma 18, that, in step-wMSO, Boolean formulæ used in if-then-else operators can be restricted to be of the form $x \in X$. Moreover, using $\sum_X$ operator, we can emulate operator $+$ and $\sum_x$ in core-wMSO. We chose to keep these operators nevertheless since they allow for an easier model of quantitative properties, without making further developments more complex.

4.2 Equivalence with weighted automata

The seminal result of Droste and Gastin (2007), linking weighted automata and wMSO formulæ, may then be rephrased as in the following theorem.

Theorem 9 For each R -weighted automaton $\mathcal{A}$ over alphabet Σ , we can effectively construct a sentence $\Phi_{\mathcal{A}}$ in core-wMSO(Σ, R), such that $\llbracket \mathcal{A} \rrbracket(w) = \llbracket \Phi_{\mathcal{A}} \rrbracket(w)$ for all $w \in \Sigma^+$.

For each sentence Φ in core-wMSO(Σ, R), we can effectively construct an R -weighted automaton $\mathcal{A}_{\Phi}$ over Σ such that $\llbracket \Phi \rrbracket(w) = \llbracket \mathcal{A}_{\Phi} \rrbracket(w)$ for all $w \in \Sigma^+$.

Since $\mathbb{N}\langle R^* \rangle$ is a semiring and the semantics $\llbracket \cdot \rrbracket$ of weighted automata and core-wMSO is the natural semantics in this semiring, one may think that Theorem 9 is a formal corollary of Droste and Gastin (2007). This is almost true but not entirely. Here we insist that the set of weights used in the formula $\Phi_{\mathcal{A}}$ is the same as the set of weights used in the weighted automaton $\mathcal{A}$ (and vice versa for $\mathcal{A}_{\Phi}$ and Φ). This is not guaranteed by Droste and Gastin (2007) though the proof can be adapted to obtain Theorem 9. We give in Sect. 4.5 a rather short proof of Theorem 9, which is based on some new ideas and which ensures that the set of weights is preserved.

The equivalence of Theorem 9 transfers to the concrete semantics without any conditions on the aggregation operator.

Corollary 10 *For each R -weighted automaton $\mathcal{A}$ over alphabet Σ , we can effectively construct a sentence $\Phi_{\mathcal{A}}$ in $\text{core-wMSO}(\Sigma, R)$, such that for all $w \in \Sigma^+$,*

$$\llbracket \mathcal{A} \rrbracket(w) = \text{aggr}(\llbracket \mathcal{A} \rrbracket(w)) = \text{aggr}(\llbracket \Phi_{\mathcal{A}} \rrbracket(w)) = \llbracket \Phi_{\mathcal{A}} \rrbracket(w).$$

For each sentence Φ in $\text{core-wMSO}(\Sigma, R)$, we can effectively construct an R -weighted automaton $\mathcal{A}_{\Phi}$ over Σ such that for all $w \in \Sigma^+$,

$$\llbracket \Phi \rrbracket(w) = \text{aggr}(\llbracket \Phi \rrbracket(w)) = \text{aggr}(\llbracket \mathcal{A}_{\Phi} \rrbracket(w)) = \llbracket \mathcal{A}_{\Phi} \rrbracket(w).$$

Instantiating the aggregation operator with aggr_{sr} for semirings, aggr_{vm} for valuation monoids, or aggr_{vs} for valuation structures, we obtain the following.

Corollary 11 *The expressive power of R -weighted automata and $\text{core-wMSO}(\Sigma, R)$ is the same in each of the following cases:*

1. $R \subseteq S$ for a semiring $(S, +, \times, 0, 1)$,
2. $R \subseteq S$ for a valuation monoid $(S, +, 0, \text{Val})$ or
3. $R \subseteq U$ for a valuation structure (U, Val, S, F) .

Before presenting the proof of Theorem 9, we show, in the next two subsections, the robustness of core-wMSO by adding some useful features in it, without changing its expressive power. Notice that the proof of equivalences is fully logical, and do not make use of translations into weighted automata: in particular, they do not require the use of Theorem 9.

4.3 Adding 0 and sum to step-wMSO

It may be convenient to use a sum operator $\Psi_1 + \Psi_2$ and constant $\mathbf{0}$ at the level of step-wMSO , as allowed for core-wMSO . We denote by $\mathbf{0}\text{-step-wMSO}(\Sigma, R)$ the fragment allowing the constant $\mathbf{0}$

$$\Psi ::= \mathbf{0} \mid r \mid \phi ? \Psi : \Psi$$

and $+\mathbf{0}\text{-step-wMSO}(\Sigma, R)$ the fragment allowing the constant $\mathbf{0}$ as well as the sum operator

$$\Psi ::= \mathbf{0} \mid r \mid \phi ? \Psi : \Psi \mid \Psi + \Psi.$$

The inductive semantics of Table 2 is enriched with

$$\llbracket \mathbf{0} \rrbracket_V(w, \sigma) = \emptyset$$

and

$$\llbracket \Psi_1 + \Psi_2 \rrbracket_V(w, \sigma) = \llbracket \Psi_1 \rrbracket_V(w, \sigma) \uplus \llbracket \Psi_2 \rrbracket_V(w, \sigma).$$

Formulae Ψ in $+\mathbf{0}\text{-step-wMSO}(\Sigma, R)$ allow for more concise specification of properties, and their semantics $\llbracket \Psi \rrbracket \in \mathbb{N}(R)$ is now a finite multiset of weights instead of simply a singleton or the empty multiset. Notice that the semantics of a formula $\prod_x \Psi$ is still well defined.

Lemma 12 *The expressive power of $\text{core-wMSO}(\Sigma, R)$ does not change if we replace $\text{step-wMSO}(\Sigma, R)$ formulae by $\mathbf{0}\text{-step-wMSO}(\Sigma, R)$ formulae.*

Proof Consider a formula Ψ of $\mathbf{0}\text{-step-wMSO}(\Sigma, R)$. We have to show that the formula $\prod_x \Psi$ can be expressed in core-wMSO . First, we construct an MSO formula ϕ_{Ψ} such that for all valid $(w, \sigma) \in \Sigma_V^+$, $\llbracket \Psi \rrbracket_V(w, \sigma) = \emptyset$ if and only if $w, \sigma \models \phi_{\Psi}$. We build ϕ_{Ψ} by induction on the formula Ψ :

$$\begin{aligned} \mathbf{0}_{\Psi} &= \top, & r_{\Psi} &= \perp \\ \text{and } \phi ? \Psi_1 : \Psi_2 &= (\phi \wedge \phi_{\Psi_1}) \vee (\neg \phi \wedge \phi_{\Psi_2}). \end{aligned}$$

It is immediate to verify the correctness of this construction.

Then, let $\tilde{\Psi}$ be the $\text{step-wMSO}(\Sigma, R)$ formula obtained from Ψ by replacing every occurrence of $\mathbf{0}$ with an arbitrary constant r of R . We can prove that for all valid $(w, \sigma) \in \Sigma_V^+$ such that $\llbracket \Psi \rrbracket_V(w, \sigma) \neq \emptyset$, we have $\llbracket \Psi \rrbracket_V(w, \sigma) = \llbracket \tilde{\Psi} \rrbracket_V(w, \sigma)$. This implies that $\prod_x \Psi$ is equivalent to the $\text{core-wMSO}(\Sigma, R)$ formula

$$(\exists x \phi_{\Psi}) ? \mathbf{0} : \prod_x \tilde{\Psi}.$$

□

Using the previous result, we finally show how to add sums in step formulae.

Lemma 13 *The expressive power of $\text{core-wMSO}(\Sigma, R)$ does not change if we replace $\text{step-wMSO}(\Sigma, R)$ formulae by $+\mathbf{0}\text{-step-wMSO}(\Sigma, R)$ formulae.*

Proof Removing sums is done in two steps. First, we can easily check that

$$\begin{aligned} \phi ? (\Psi_1 + \Psi_2) : \Psi_3 &\equiv (\phi ? \Psi_1 : \Psi_3) + (\phi ? \Psi_2 : \Psi_3) \\ \phi ? \Psi_1 : (\Psi_2 + \Psi_3) &\equiv (\phi ? \Psi_1 : \Psi_2) + (\phi ? \mathbf{0} : \Psi_3). \end{aligned}$$

Hence, for each formula Ψ of $+\mathbf{0}\text{-step-wMSO}(\Sigma, R)$ we can construct an equivalent formula $\Psi_1 + \dots + \Psi_n$ where the Ψ_i are in $\mathbf{0}\text{-step-wMSO}(\Sigma, R)$.

For the second step, we let $\bar{X} = (X_1, \dots, X_n)$ be a tuple of second-order variables. We simply write $\sum_{\bar{X}}$ instead of $\sum_{X_1} \dots \sum_{X_n}$. We also let $\text{partition}(\bar{X})$ be an MSO formula stating that the sets associated with variables $X_1, \dots, X_n$ form a partition of the positions. Let

$\Psi' = x \in X_1 ? \Psi_1 : \dots x \in X_n ? \Psi_n : \mathbf{0}$. Notice that Ψ' is in **0-step-wMSO**(Σ, R). We claim that the **core-wMSO** formulae

$$\Phi = \prod_x (\Psi_1 + \dots + \Psi_n)$$

$$\Phi' = \sum_{\bar{X}} \text{partition}(\bar{X}) ? [\prod_x \Psi'] : \mathbf{0}$$

are equivalent. The proof that $\Phi \equiv \Phi'$ relies on the distributivity of the semiring $\mathbb{N}\langle R^* \rangle$. Details are left to the reader. We finally obtain the result by applying Lemma 12 to the formula $\prod_x \Psi'$. $\square$

4.4 Adding new binary operators

For certain modelling applications, it might be useful to enhance our core logic with extra operators, for instance binary operators. In particular, the classical **wMSO** logic (see Sect. 5) allows a quantitative extension of the conjunction operator, though in a restricted manner. Extra operators also allow to define quantitative properties compositionally.

Example 14 Let us denote by Φ_a the **core-wMSO** formula of Example 7 computing the maximal length of blocks of a 's when concretely evaluated in the semiring $(\mathbb{N} \cup \{-\infty\}, \max, +, -\infty, 0)$. If we want to compute the sum of the maximal length of blocks of a 's and of the maximal length of blocks of b 's in a word, it seems natural to authorise the use of the binary sum to decompose this objective in two distinct ones. Then, we could use formula $\Phi_a \diamond \Phi_b$ (where Φ_b is obtained by inverting the roles of a and b in Φ_a), denoting by $\diamond$ the new binary operator allowing to compute binary sums.

We now show how to add binary operators to the logic without changing its expressive power. Indeed, we can proceed in the same way to add operators with other arities.

Let $\diamond : R^2 \rightarrow R$ be an arbitrary binary operation over R . It naturally extends to a partial binary operation over R^* by setting for all sequences $\alpha = r_1 \dots r_n$ and $\alpha' = r'_1 \dots r'_n$ of same length

$$\alpha \diamond \alpha' = (r_1 \diamond r'_1) \dots (r_n \diamond r'_n).$$

The operator is not defined on sequences of different lengths. Then, as explained in the preliminaries, this binary operation extends to finite multisets in $\mathbb{N}\langle R \rangle$ or in $\mathbb{N}\langle R^* \rangle$.

We enrich the syntax defined in Table 1 with $\Psi_1 \diamond \Psi_2$ for **step-wMSO** and with $\Phi_1 \diamond \Phi_2$ for **core-wMSO**. The semantics defined in Tables 2 and 3 is also enriched with

$$\llbracket \Psi_1 \diamond \Psi_2 \rrbracket_V(w, \sigma) = \llbracket \Psi_1 \rrbracket_V(w, \sigma) \diamond \llbracket \Psi_2 \rrbracket_V(w, \sigma)$$

$$\llbracket \Phi_1 \diamond \Phi_2 \rrbracket_V(w, \sigma) = \llbracket \Phi_1 \rrbracket_V(w, \sigma) \diamond \llbracket \Phi_2 \rrbracket_V(w, \sigma).$$

Example 15 When $\diamond$ is interpreted as $+$, the formula $\Phi_a \diamond \Phi_b$ of Example 14 has the following abstract semantics over $w = baaba$:

$$\llbracket \Phi_a \rrbracket(w) = \{00000, 01100, 00001\}$$

$$\llbracket \Phi_b \rrbracket(w) = \{10000, 00010, 00000\}$$

$$\llbracket \Phi_a \diamond \Phi_b \rrbracket(w) = \{10000, 00010, 00000, 11100, 01110, 01100, 10001, 00011, 00001\}.$$

Notice that its concrete semantics in the semiring $(\mathbb{N} \cup \{-\infty\}, \max, +, -\infty, 0)$ is 3 which is indeed the sum of the maximal length of blocks of a 's and b 's.

We now show that adding such a binary operator does not increase the expressive power of **step-wMSO** and **core-wMSO**, respectively. To that extent, let us denote by $\diamond$ -**step-wMSO** the logic **step-wMSO** enriched with the diamond operator. We also write $\diamond$ -**core-wMSO** for **core-wMSO** enriched with the diamond operator.

Proposition 16 *The expressive power of step-wMSO (respectively, core-wMSO) does not change if we add formulae $\Psi_1 \diamond \Psi_2$ in step-wMSO (respectively, $\Phi_1 \diamond \Phi_2$ in core-wMSO).*

Proof Let Ψ_1, Ψ_2, Ψ_3 be $\diamond$ -**step-wMSO** formulae. By case splitting, it is clear that

$$(\varphi ? \Psi_1 : \Psi_2) \diamond \Psi_3 \equiv \varphi ? (\Psi_1 \diamond \Psi_3) : (\Psi_2 \diamond \Psi_3)$$

$$\Psi_3 \diamond (\varphi ? \Psi_1 : \Psi_2) \equiv \varphi ? (\Psi_3 \diamond \Psi_1) : (\Psi_3 \diamond \Psi_2).$$

By applying inductively the above equivalences, we can rewrite any $\diamond$ -**step-wMSO** formula until the $\diamond$ operator is only applied to constants from R . Now, for $r_1, r_2 \in R$, we have $r_1 \diamond r_2 \in R$ which is a legal **step-wMSO** formula. Therefore, we have proved that any formula in $\diamond$ -**step-wMSO** can be rewritten in a semantically equivalent **step-wMSO** formula.

Since $\diamond$ distributes over $\uplus$ in the semiring $\mathbb{N}\langle R^* \rangle$, we deduce that $\diamond$ distributes (left and right) over the sum operators of **core-wMSO** (we only give equations for left distributivity below):

$$\Phi_1 \diamond (\Phi_2 + \Phi_3) \equiv (\Phi_1 \diamond \Phi_2) + (\Phi_1 \diamond \Phi_3)$$

$$\Phi_1 \diamond \sum_x \Phi_2 \equiv \sum_x (\Phi_1 \diamond \Phi_2)$$

$$\Phi_1 \diamond \sum_X \Phi_2 \equiv \sum_X (\Phi_1 \diamond \Phi_2).$$

By case splitting, we also get as above that $\diamond$ distributes over the if-then-else operator. Note also that $\mathbf{0} \diamond \Phi \equiv \mathbf{0} \equiv \Phi \diamond \mathbf{0}$. Hence, applying inductively the above equivalences, we can rewrite any $\diamond$ -**core-wMSO** formula until the $\diamond$ operator is only applied to products $(\prod_x \Psi) \diamond (\prod_x \Psi')$ (up to renaming, we may assume that the same variable x is used in both products). We claim that

$$(\prod_x \Psi) \diamond (\prod_x \Psi') \equiv \prod_x (\Psi \diamond \Psi'). \quad (2)$$

Let $w \in \Sigma^+$ be a word of length n and σ be a valuation of the variables in $V \supseteq (\text{free}(\Psi) \cup \text{free}(\Psi')) \setminus \{x\}$ (the semantic equivalence is trivially verified on invalid encodings $\bar{w} \in \Sigma_V^+$). Since Ψ and Ψ' are **step-wMSO** formulæ, for all $1 \leq i \leq n$, we have

$$\begin{aligned} A_i &= \llbracket \Psi \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i]) \in \mathbb{N}\langle R \rangle \\ B_i &= \llbracket \Psi' \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i]) \in \mathbb{N}\langle R \rangle \\ A_i \diamond B_i &= \llbracket \Psi \diamond \Psi' \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i]) \in \mathbb{N}\langle R \rangle \end{aligned}$$

and these multisets are singletons. Using this fact, we can easily check that³

$$(A_1 \dots A_n) \diamond (B_1 \dots B_n) = (A_1 \diamond B_1) \dots (A_n \diamond B_n). \quad (3)$$

We obtain (2) since

$$\begin{aligned} \llbracket \prod_x \Psi \rrbracket_V(w, \sigma) &= A_1 \dots A_n \\ \llbracket \prod_x \Psi' \rrbracket_V(w, \sigma) &= B_1 \dots B_n \\ \llbracket \prod_x \Psi \diamond \Psi' \rrbracket_V(w, \sigma) &= (A_1 \diamond B_1) \dots (A_n \diamond B_n). \end{aligned}$$

This concludes the proof since $\Psi \diamond \Psi'$ can be translated into an equivalent **step-wMSO** formula. $\square$

Consider the concrete semantics $\llbracket \cdot \rrbracket = \text{aggr}(\llbracket \cdot \rrbracket)$ defined with an aggregation $\text{aggr}: \mathbb{N}\langle R^* \rangle \rightarrow S$. Assume also that $\diamond: S^2 \rightarrow S$ is defined on S . We say that $\diamond$ distributes over **aggr** if for all $n > 0$ and $A, B \in \mathbb{N}\langle R^n \rangle$, we have

$$\text{aggr}(A \diamond B) = \text{aggr}(A) \diamond \text{aggr}(B).$$

In this case, we can easily check that the concrete semantics is compositional: for all $\bar{w} \in \Sigma_V^+$, we have $\llbracket \Phi_1 \diamond \Phi_2 \rrbracket(\bar{w}) = \llbracket \Phi_1 \rrbracket(\bar{w}) \diamond \llbracket \Phi_2 \rrbracket(\bar{w})$.

Example 17 In the case of a semiring $(S, +, \times, 0, 1)$, the aggregation is $\sum \prod$ (as defined in Example 3). We may easily check that for commutative semirings, the concrete semantics is compositional with respect to the product $\times$.

4.5 Proof of Theorem 9

We show first that we can restrict **step-wMSO** to a smaller fragment without changing the expressive power of **core-wMSO**. The syntax of **set-step-wMSO**(R) is given by

$$\Psi ::= r \mid x \in X ? \Psi : \Psi \quad (\text{set-step-wMSO}),$$

where x is a fixed first-order variable, X ranges over monadic second-order variables and r ranges over R . This restriction has also been considered previously. For instance, in

³ Actually, (3) holds for arbitrary multisets $A_i, B_i \in \mathbb{N}\langle R \rangle$.

the weighted timed setting (Droste and Perevoshchikov 2014), Droste and Perevoshchikov translate weighted (timed) automata into weighted sentences where Boolean formulæ inside the universal quantification are of the form $x \in X$ only. In our context, it means only **set-step-wMSO** inside a product $\prod_x$. Similarly, in the context of trees (Fülöp 2012), Fülöp, Stüber, and Vogler use an operation $H(\omega)$ which renames every node of the input tree with an operator from some family ω (coming from a multi-operator monoid). Again, the renaming is described by means of formulæ of the form $x \in X$ only, and not by more general **MSO** formulæ.

Lemma 18 *The expressive power of core-wMSO(Σ, R) does not change if we replace step-wMSO(Σ, R) formulæ by set-step-wMSO(R) formulæ.*

Proof We start with a **core-wMSO** formula $\Phi = \prod_x \Psi$, where Ψ is a **step-wMSO**(Σ, R) formula. Let $\varphi_1, \dots, \varphi_n$ be the **MSO** formulæ occurring in Ψ as conditions of the if-then-else operator. We let $\bar{X} = (X_1, \dots, X_n)$ be a tuple of fresh second-order variables. Let also Ψ' be the formula obtained from Ψ by replacing every occurrence of φ_i by $x \in X_i$, for all $1 \leq i \leq n$. Notice that Ψ' is a **set-step-wMSO**(R) formula. We claim that $\Phi = \prod_x \Psi$ is equivalent to the formula

$$\Phi' = \sum_{\bar{X}} \left(\bigwedge_i \forall x (x \in X_i \leftrightarrow \varphi_i) \right) ? \left(\prod_x \Psi' \right) : 0.$$

Indeed, let $V = \text{free}(\Phi) = \text{free}(\prod_x \Psi)$ and $V' = V \cup \{X_1, \dots, X_n\}$. For every valid $(w, \sigma) \in \Sigma_V^+$, there is a unique $(w, \sigma') \in \Sigma_{V'}^+$ such that $\sigma'|_V = \sigma$ and $w, \sigma' \models \bigwedge_i \forall x (x \in X_i \leftrightarrow \varphi_i)$. For all $1 \leq i \leq n$, we have $\sigma'(X_i) = \{j \in \text{pos}(w) \mid w, \sigma[x \mapsto j] \models \varphi_i\}$. We obtain $\llbracket \Phi' \rrbracket(w, \sigma) = \llbracket \prod_x \Psi' \rrbracket(w, \sigma')$. Then, it is easy to check by induction on Ψ that for all $j \in \text{pos}(w)$ we have $\llbracket \Psi \rrbracket(w, \sigma[x \mapsto j]) = \llbracket \Psi' \rrbracket(w, \sigma'[x \mapsto j])$. We deduce that $\llbracket \Phi' \rrbracket(w, \sigma) = \llbracket \prod_x \Psi' \rrbracket(w, \sigma') = \llbracket \prod_x \Psi \rrbracket(w, \sigma) = \llbracket \Phi \rrbracket(w, \sigma)$ for all valid $(w, \sigma) \in \Sigma_V^+$. $\square$

Proof of Theorem 9 Let $\mathcal{A} = (Q, \Delta, \text{wgt}, I, F)$ be a weighted automaton. We use a set variable X_δ for each transition $\delta \in \Delta$ and we let $\bar{X} = (X_\delta)_{\delta \in \Delta}$. Intuitively, the tuple $\bar{X}$ encodes a run of $\mathcal{A}$ over a word w when each set variable X_δ is interpreted as the set of positions at which transition δ is used in that run.

We can easily write an **MSO** formula $\text{run}(\bar{X})$ which evaluates to true on some word w if and only if $\bar{X}$ encodes a run of $\mathcal{A}$ on w starting from I and ending in F . First, we state that $\bar{X}$ is a partition on the positions of w . Then we request that if the first position of w is in X_δ then $\delta \in I \times \Sigma \times Q$ is initial. Similarly, the transition of the last position should be final. Finally, if $\delta = (p, a, q)$ and $\delta' = (p', a', q')$ are the transitions of two consecutive positions of w then $q = p'$. It is routine to write all these requirements in **MSO** (even in **FO**₃).

Assuming that $\text{run}(\bar{X})$ holds, we let $\text{weight}(x, \bar{X})$ be the **set-step-wMSO** formula which evaluates to $\text{wgt}(\delta)$ where $\delta \in \Delta$ is the unique transition such that $x \in X_\delta$. Formally, if $\Delta = \{\delta_1, \delta_2, \dots, \delta_n\}$ then we define $\text{weight}(x, \bar{X})$ as

$$x \in X_{\delta_1} ? \text{wgt}(\delta_1) : \dots x \in X_{\delta_{n-1}} ? \text{wgt}(\delta_{n-1}) : \text{wgt}(\delta_n)$$

and

$$\Phi_{\mathcal{A}} = \sum_{\bar{X}} \text{run}(\bar{X}) ? (\prod_x \text{weight}(x, \bar{X})) : \mathbf{0}.$$

We can easily check that for all words $w \in \Sigma^+$ we have $\|\mathcal{A}\|(w) = \|\Phi\|(w)$.

Conversely, we proceed by induction on Φ , hence we have to deal with free variables. So we construct for each formula Φ a weighted automaton $\mathcal{A}_\Phi$ over the alphabet $\Sigma_{\text{free}(\Phi)} = \Sigma \times \{0, 1\}^{\text{free}(\Phi)}$ such that for all $\bar{w} \in \Sigma_{\text{free}(\Phi)}^+$ we have $\|\Phi\|(\bar{w}) = \|\mathcal{A}_\Phi\|(\bar{w})$.

It is folklore that we may increase the set of variables encoded in the alphabet whenever needed, e.g. to deal with sum or if-then-else. Formally, if $V \subseteq V'$ then we can lift an automaton $\mathcal{A}_V$ defined on the alphabet Σ_V to an automaton $\mathcal{A}_{V'}$ defined on $\Sigma_{V'}$ such that for all valid $(w, \sigma) \in \Sigma_{V'}^+$ we have $\|\mathcal{A}_{V'}\|(w, \sigma) = \|\mathcal{A}_V\|(w, \sigma|_V)$.

The automaton $\mathcal{A}_\mathbf{0}$ has a single state which is initial but not final and has no transitions.

We recall the classical constructions for the additive operators of **core-wMSO**: $+$, $\sum_x$ and $\sum_X$.

If $\Phi = \Phi_1 + \Phi_2$ then $\mathcal{A}_\Phi$ is obtained as the disjoint union of $\mathcal{A}_{\Phi_1}$ and $\mathcal{A}_{\Phi_2}$, both lifted to Σ_Φ .

If $\Phi = \sum_X \Phi_1$ then $\mathcal{A}_\Phi$ is obtained via a variant ⁴ of the projection construction starting from $\mathcal{A}_{\Phi_1}$. Assume that $\mathcal{A}_{\Phi_1} = (Q, \Delta, \text{wgt}, I, F)$. We define $\mathcal{A}_\Phi = (Q \times \{0, 1\}, \Delta', \text{wgt}', I \times \{0\}, F \times \{0, 1\})$ over alphabet $\Sigma_{\text{free}(\Phi)}$ by letting

$$((p, i), a, (q, j)) \in \Delta' \text{ iff } (p, (a, j), q) \in \Delta$$

where (a, j) denotes the letter in $\Sigma_{\text{free}(\Phi) \cup \{X\}}$ where the value of the X -component is given by j and the remaining $\Sigma_{\text{free}(\Phi)}$ -components (different from X) are given by a . We also let

$$\text{wgt}'((p, i), a, (q, j)) = \text{wgt}(p, (a, j), q).$$

This transfer of the alphabet component for X to the state of $\mathcal{A}_\Phi$ allows us to define a bijection between the accepting runs of $\mathcal{A}_{\Phi_1}$ and the accepting runs of $\mathcal{A}_\Phi$, preserving sequences

⁴ As already noticed in [Droste and Meinecke \(2012\)](#), a simple projection does not work. Indeed, it would result in transition labels that are multisets of weights, which is not possible since our theorem requires the same set of weights for the automaton and the formula.

of weights. Then, we deduce easily that $\|\mathcal{A}_\Phi\| = \|\Phi\|$ over alphabet $\Sigma_{\text{free}(\Phi)}$.

If $\Phi = \sum_x \Phi_1$, the construction is almost the same. In the definition of $\mathcal{A}_\Phi$, the set of accepting states is $F \times \{1\}$ and the transitions are given by

$$((p, 0), a, (q, j)) \in \Delta' \text{ iff } (p, (a, j), q) \in \Delta$$

$$((p, 1), a, (q, 1)) \in \Delta' \text{ iff } (p, (a, 0), q) \in \Delta$$

with weights inherited as before

$$\text{wgt}'((p, 0), a, (q, j)) = \text{wgt}(p, (a, j), q)$$

$$\text{wgt}'((p, 1), a, (q, 1)) = \text{wgt}(p, (a, 0), q).$$

We turn now to the more interesting cases: if-then-else and $\prod_x$. Noticed that $\phi ? \Phi_1 : \Phi_2$ is equivalent to $(\phi ? \Phi_1 : \mathbf{0}) + (\neg\phi ? \Phi_2 : \mathbf{0})$, hence we only need to construct an automaton for $\Phi = \phi ? \Phi_1 : \mathbf{0}$. Let $V = \text{free}(\Phi) = \text{free}(\phi) \cup \text{free}(\Phi_1)$. Since ϕ is a (Boolean) **MSO** formula, by [Büchi \(1960\)](#); [Elgot \(1961\)](#); [Trakhtenbrot \(1961\)](#), we can construct a deterministic⁵ automaton $\mathcal{A}_\phi$ over the alphabet Σ_V which accepts a word $\bar{w} \in \Sigma_V^+$ if and only if it is a valid encoding $\bar{w} = (w, \sigma)$ satisfying ϕ . Now, by induction, we have an automaton $\mathcal{A}_{\Phi_1}$ over Σ_V which is equivalent to Φ_1 : for all $\bar{w} \in \Sigma_V^+$, $\|\Phi_1\|(\bar{w}) = \|\mathcal{A}_{\Phi_1}\|(\bar{w})$. The automaton $\mathcal{A}_\Phi$ is obtained as the “intersection” of $\mathcal{A}_\phi$ and $\mathcal{A}_{\Phi_1}$ (see the formal construction below). Now, let $\bar{w} \in \Sigma_V^+$. If $\bar{w}$ is not valid or $\bar{w} = (w, \sigma)$ is valid and does not satisfy ϕ then $\mathcal{A}_\phi$ (hence also $\mathcal{A}_\Phi$) has no accepting run on $\bar{w}$ and we obtain $\|\mathcal{A}_\Phi\|(\bar{w}) = \emptyset = \|\Phi\|(\bar{w})$. On the other hand, assume that $\bar{w} = (w, \sigma)$ is valid and satisfy ϕ . Since $\mathcal{A}_\phi$ is deterministic, there is a bijection between the accepting runs of $\mathcal{A}_\Phi$ and the accepting runs of $\mathcal{A}_{\Phi_1}$. By construction of $\mathcal{A}_\Phi$, this bijection preserves the sequence of weights associated with a run. We deduce that $\|\mathcal{A}_\Phi\|(w, \sigma) = \|\mathcal{A}_{\Phi_1}\|(w, \sigma) = \|\Phi\|(w, \sigma)$.

We give now the formal definition of $\mathcal{A}_\Phi$. Let $\mathcal{A}_{\Phi_1} = (Q_1, \Delta_1, \text{wgt}_1, I_1, F_1)$ be the weighted automaton over Σ_V for Φ_1 . Let $\mathcal{A}_\phi = (Q_2, \Delta_2, I_2, F_2)$ be the deterministic automaton over Σ_V for ϕ . Then, we define $\mathcal{A}_\Phi = (Q, \Delta, \text{wgt}, I, F)$ with $Q = Q_1 \times Q_2$, $I = I_1 \times I_2$, $F = F_1 \times F_2$, Δ is the set of triples $\delta = ((p_1, p_2), a, (q_1, q_2))$ such that $\delta_1 = (p_1, a, q_1) \in \Delta_1$ and $(p_2, a, q_2) \in \Delta_2$, and $\text{wgt}(\delta) = \text{wgt}(\delta_1)$.

Finally, it remains to deal with the case $\Phi = \prod_x \Psi$. By [Lemma 18](#), we may assume that Ψ is a formula in **set-step-wMSO**(R). So $\text{free}(\Psi) = \{x, X_1, \dots, X_n\}$ and the tests in Ψ are of the form $x \in X_i$ for some $i \in \{1, \dots, n\}$. Also, $\text{free}(\Phi) = \{X_1, \dots, X_n\}$ consists of second-order variables only, so every word $\bar{w} \in \Sigma_{\text{free}(\Phi)}^+$ is valid.

⁵ We could also use an unambiguous automaton for $\mathcal{A}_\phi$.

For every $\tau \in \{0, 1\}^n$, we define the evaluation $\Psi(\tau)$ inductively as follows: $r(\tau) = r$ and

$$(x \in X_i ? \Psi_1 : \Psi_2)(\tau) = \begin{cases} \Psi_1(\tau) & \text{if } \tau_i = 1 \\ \Psi_2(\tau) & \text{otherwise.} \end{cases}$$

Let $\bar{w} = (a_1, \tau_1) \dots (a_k, \tau_k) \in \Sigma_{\text{free}(\Phi)}^+$ with $a_j \in \Sigma$ and $\tau_j \in \{0, 1\}^{\text{free}(\Phi)}$ for all $1 \leq j \leq k$. We can easily check that $\llbracket \Phi \rrbracket(\bar{w}) = \{\Psi(\tau_1) \dots \Psi(\tau_k)\}$. Define $\mathcal{A}_\Phi = (Q, \Delta, \text{wgt}, I, F)$ with a single state which is both initial and final ($Q = I = F = \{q\}$) and for every $a \in \Sigma$ and $\tau \in \{0, 1\}^{\text{free}(\Phi)}$, there is a transition $\delta = (q, (a, \tau), q) \in \Delta$ with $\text{wgt}(\delta) = \Psi(\tau)$. It is clear that for every word $\bar{w} = (a_1, \tau_1) \dots (a_k, \tau_k) \in \Sigma_{\text{free}(\Phi)}^+$, the automaton $\mathcal{A}_\Phi$ has a single run on $\bar{w}$ whose sequence of weights is $\Psi(\tau_1) \dots \Psi(\tau_k)$. Therefore, $\llbracket \mathcal{A}_\Phi \rrbracket(\bar{w}) = \llbracket \Phi \rrbracket(\bar{w})$, which concludes the proof. $\square$

5 Restricted weighted MSO logic

We now present the syntax and semantics of the full **wMSO** logic that has been studied over semirings (Droste and Gastin 2007), valuation monoids (Droste and Meinecke 2012) and valuation structures (Droste and Perevoshchikov 2013). The syntax used in these previous works is different. Also, there is no separate semantics for the Boolean fragment, instead, it is obtained as a special case of the quantitative semantics. As we will see, this choice requires some additional conditions on the weight domain, called hypothesis (01) below. In order to obtain the same expressive power as weighted automata, we also have to restrict the usage of conjunction and universal quantifications in **wMSO**.

We present effective translations in both directions relating restricted **wMSO** with **core-wMSO**, and the conditions that the weight domain has to fulfil in different settings. Using Corollary 11, we obtain a purely logical proof of the equivalence between restricted **wMSO** and weighted automata, using **core-wMSO** as an intermediary, simple and elegant, logical formalism.

5.1 wMSO over semirings and valuation monoids

For a set R of weights, the logic **wMSO**(Σ, R) studied in Droste and Gastin (2007) and Droste and Meinecke (2012) is given by the following grammar:

$$\Xi ::= r \mid \phi \mid \Xi \vee \Xi \mid \Xi \wedge \Xi \mid \exists x \Xi \mid \forall x \Xi \mid \exists X \Xi,$$

where x and y are first-order variables, X is a second-order variable, $\phi \in \text{MSO}(\Sigma)$ and $r \in R$.

Table 4 Semantics of **wMSO** over product valuation monoids

$\llbracket r \rrbracket_V(w, \sigma) = r$
$\llbracket \beta \rrbracket_V(w, \sigma) = \begin{cases} 1 & \text{if } w, \sigma \models \beta \\ 0 & \text{otherwise} \end{cases}$
$\llbracket \neg \phi \rrbracket_V(w, \sigma) = \begin{cases} 1 & \text{if } \llbracket \phi \rrbracket_V(w, \sigma) = 0 \\ 0 & \text{otherwise} \end{cases}$
$\llbracket \xi \vee \xi' \rrbracket_V(w, \sigma) = \llbracket \xi \rrbracket_V(w, \sigma) + \llbracket \xi' \rrbracket_V(w, \sigma)$
$\llbracket \xi \wedge \xi' \rrbracket_V(w, \sigma) = \llbracket \xi \rrbracket_V(w, \sigma) \diamond \llbracket \xi' \rrbracket_V(w, \sigma)$
$\llbracket \exists x \xi \rrbracket_V(w, \sigma) = \sum_{i \in \text{pos}(w)} \llbracket \xi \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i])$
$\llbracket \forall x \xi \rrbracket_V(w, \sigma) = \text{Val}((\llbracket \xi \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i]))_{i \in \text{pos}(w)})$
$\llbracket \exists X \xi \rrbracket_V(w, \sigma) = \sum_{I \subseteq \text{pos}(w)} \llbracket \xi \rrbracket_{V \cup \{X\}}(w, \sigma[X \mapsto I])$
$\llbracket \forall X \phi \rrbracket_V(w, \sigma) = \text{Val}((\llbracket \phi \rrbracket_{V \cup \{X\}}(w, \sigma[X \mapsto I]))_{I \subseteq \text{pos}(w)})$

β stands for an atomic formula among $P_a(x), x \leq y, x \in X$

Defining the semantics of the conjunction operator requires the introduction of an additional binary operator $\diamond$. Expressing the semantics of Boolean formulae with the quantitative semantics of weighted formulae requires special elements 0 and 1. A semiring is naturally equipped with such objects, the multiplication and its zero and unit elements. A valuation monoid equipped with such objects is called a product valuation monoid. More formally, a product valuation monoid is a tuple $(S, +, 0, \text{Val}, \diamond, 1)$ with $(S, +, 0)$ a commutative monoid, $\text{Val}: S^* \rightarrow S, \diamond: S^2 \rightarrow S$, and $1 \in S$. In Droste and Meinecke (2012), additional conditions are given in the definition of product valuation monoids. We will highlight these conditions in the following. Semirings are a special case of product valuation monoids. In this section, we assume that $\{0, 1\} \subseteq R \subseteq S$ is a subset of a product valuation monoid $(S, +, 0, \text{Val}, \diamond, 1)$.

The semantics of a Boolean formula $\xi = \phi$ or a weighted formula $\xi = \Xi$ is defined uniformly. Let $\bar{w} \in \Sigma_V^+$ with $\text{free}(\xi) \subseteq V$. We define $\llbracket \xi \rrbracket_V(\bar{w}) = 0$ if $\bar{w}$ is not valid. If $\bar{w}$ is the valid encoding of (w, σ) then the semantics is given in Table 4.

To relate **wMSO** with **core-wMSO**, we introduce an intermediary logic **wMSO'** which semantically separates the Boolean fragment from the quantitative one. The syntax of **wMSO'**(Σ, R) is given by the grammar

$$\Xi ::= r \mid \phi ? \Xi : \Xi \mid \Xi \vee \Xi \mid \Xi \wedge \Xi \mid \exists x \Xi \mid \forall x \Xi \mid \exists X \Xi$$

where x and y are first-order variables, X is a second-order variable, $\phi \in \text{MSO}(\Sigma)$ and $r \in R$. Notice that a Boolean formula is no more a weighted formula. Instead, the if-then-else construct is used in **wMSO'**.

The semantics of **wMSO'** is defined as in Table 4, removing the cases $\beta, \neg \phi$ and $\forall X \phi$, restricting the other cases to weighted formulae $\xi = \Xi$, and using the case-splitting semantics for the if-then-else operator:

$$\llbracket \phi ? \Xi : \Xi' \rrbracket_V(w, \sigma) = \begin{cases} \llbracket \Xi \rrbracket_V(w, \sigma) & \text{if } w, \sigma \models \phi \\ \llbracket \Xi' \rrbracket_V(w, \sigma) & \text{otherwise.} \end{cases}$$

Let us now give a list of properties that the product valuation monoid should fulfil in order for $\mathbf{wMSO}$ and $\mathbf{wMSO}'$ to be expressively equivalent:

- $0 \diamond s = s \diamond 0 = 0$ for all $s \in S$,
- $1 \diamond s = s \diamond 1 = s$ for all $s \in S$,
- $\text{Val}(s_1 \dots s_n) = 0$ for all $s_1 \dots s_n \in S^+$ such that $s_i = 0$ for some i ,
- and $\text{Val}(1 \dots 1) = 1$.

In the following, we call this list of hypotheses **(01)**.

Lemma 19 *Under hypothesis (01), $\mathbf{wMSO}(\Sigma, R)$ and $\mathbf{wMSO}'(\Sigma, R)$ have the same expressive power.*

Proof We first show by induction that, if $\phi \in \mathbf{MSO}$ is a Boolean formula then the semantics of $\mathbf{wMSO}$ gives for a valid word $\bar{w} = (w, \sigma)$

$$\llbracket \phi \rrbracket_V(w, \sigma) = \begin{cases} 1 & \text{if } w, \sigma \models \phi \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

The property holds for atomic formulae by definition of the semantics in Table 4. It is also trivial by induction for negation. For conjunction, the property follows from the first two items of hypothesis **(01)** (only used with $s \in \{0, 1\}$), and for the universal quantifications, it follows from the last two items.

Then, the translation of $\Xi \in \mathbf{wMSO}$ to $\Xi' \in \mathbf{wMSO}'$ only replaces maximal Boolean subformulae ϕ in Ξ with $\phi ? 1 : 0$. An occurrence of a Boolean subformula is maximal in Ξ if it is not a strict subformula of another Boolean subformula in Ξ . The equality between both semantics is a direct consequence of (4).

In the other direction, we simply replace formulae $\phi ? \Xi_1 : \Xi_2$ with $(\phi \wedge \Xi_1) \vee (\neg \phi \wedge \Xi_2)$. Then, the equality between both semantics is a consequence of (4) and the first two items of hypothesis **(01)**, used here with all possible values of s . $\square$

In Droste and Meinecke (2012), several fragments of $\mathbf{wMSO}$ are studied. For instance, the set of almost Boolean formulae of $\mathbf{wMSO}$ is the fragment containing all constants $r \in R$, all Boolean formulae ϕ , and which is closed under disjunction and conjunction. In contrast, in $\mathbf{wMSO}'$, we define step formulae as the fragment containing all constants $r \in R$ and closed under the if-then-else operator. Notice that step formulae of $\mathbf{wMSO}'$ correspond to step- $\mathbf{wMSO}$ formulae. We now show the relationship between the two fragments.

Lemma 20 *For every almost Boolean formula Ξ of $\mathbf{wMSO}(\Sigma, R)$, we can construct an equivalent step formula Ξ' of $\mathbf{wMSO}'(\Sigma, S)$. Conversely, under hypothesis (01), every step formula Ξ' of $\mathbf{wMSO}'(\Sigma, R)$ can be translated into an equivalent almost Boolean formula Ξ of $\mathbf{wMSO}(\Sigma, R)$.*

Proof Let $\{\phi_1, \dots, \phi_n\}$ be the set of maximal $\mathbf{MSO}$ formulae occurring in Ξ . The proof is by induction on n .

When $n = 0$, Ξ is a positive Boolean combination of constants. Replacing $\wedge$ with $\diamond$ and $\vee$ with $+$, we obtain an expression which evaluates to a new constant $s \in S$ (not necessarily in R). Then, $\Xi' = s$ is a step formula which is equivalent to Ξ .

Assume now that $n > 0$. Consider the formulae $\Xi[\phi_n/1]$ and $\Xi[\phi_n/0]$ obtained by substituting maximal occurrences of ϕ_n with 1 and 0, respectively. These are almost Boolean formulae with $n - 1$ maximal Boolean formula. By induction, there are equivalent step formulae Ξ'_1 and Ξ'_0 of $\mathbf{wMSO}'$. Moreover, by (4), we can show by induction that for a valid word $\bar{w} = (w, \sigma)$

$$\llbracket \Xi \rrbracket_V(w, \sigma) = \begin{cases} \llbracket \Xi[\phi_n/1] \rrbracket_V(w, \sigma) & \text{if } w, \sigma \models \phi_n \\ \llbracket \Xi[\phi_n/0] \rrbracket_V(w, \sigma) & \text{otherwise.} \end{cases}$$

Then, formula $\Xi' = \phi_n ? \Xi'_1 : \Xi'_0$ is a step formula equivalent to Ξ .

Conversely, the translation from $\mathbf{wMSO}'$ to $\mathbf{wMSO}$ shown in Lemma 19, if applied to a step formula, produces an almost Boolean formula. $\square$

Notice that the translation from almost Boolean formulae in $\mathbf{wMSO}$ to step formulae in $\mathbf{wMSO}'$, not only modifies the set of constants used in the logic, but also expands exponentially the size of the formulae.

Other fragments of $\mathbf{wMSO}$ are studied in Droste and Meinecke (2012). A formula $\Xi \in \mathbf{wMSO}$ is $\forall$ -restricted if every subformula $\forall x \xi$ is such that ξ is almost Boolean. The formula Ξ is $\wedge$ -restricted⁶ if every subformula $\xi \wedge \xi'$ is such that both ξ and ξ' are almost Boolean, or ξ or ξ' is Boolean.

We will relate these fragments with corresponding ones in $\mathbf{wMSO}'$. A formula $\Xi \in \mathbf{wMSO}'$ is $\forall$ -restricted if every subformula $\forall x \xi$ is such that ξ is a step formula. Formula Ξ is $\wedge$ -restricted if $\wedge$ is used only in Boolean subformulae.

Remark 21 Under hypothesis **(01)**, thanks to the translation of Lemma 19, $\forall$ -restricted (respectively, $\wedge$ -restricted) formulae of $\mathbf{wMSO}'$ translate into equivalent $\forall$ -restricted (respectively, $\wedge$ -restricted) formulae of $\mathbf{wMSO}$. Conversely, using the translation of Lemma 20 for almost Boolean formulae occurring just below a universal quantification, we can translate $\forall$ -restricted formulae of $\mathbf{wMSO}$ into $\forall$ -restricted formulae of $\mathbf{wMSO}'$. Moreover, starting from a $\wedge$ -restricted formula Ξ of $\mathbf{wMSO}$, we first apply the transformation of Lemma 20 to maximal almost Boolean formulae which are not Boolean. We obtain a new formula Ξ' in which the

⁶ These formulae are called strongly $\wedge$ -restricted in Droste and Meinecke (2012), in contrast with a slightly less restrained fragment of $\wedge$ -restricted formulae.

remaining conjunctions which are not inside a Boolean subformula are of the form $\phi \wedge \Xi_1$ or $\Xi_1 \wedge \phi$ with ϕ a Boolean formula. We translate these formulae into $\phi ? \Xi_1 : 0$. We may still have some maximal Boolean subformulae φ which are not guards of an if-then-else operator. We replace these with $\varphi ? 1 : 0$. We obtain an equivalent $\wedge$ -restricted formula Ξ'' of $\mathbf{wMSO}'$.

We now establish the relationship between $\mathbf{core-wMSO}$ and these fragments of $\mathbf{wMSO}'$. The first direction does not require any additional hypotheses.

The semantic equivalence between a formula $\Phi \in \mathbf{core-wMSO}$ and a formula Ξ in $\mathbf{wMSO}'$ or $\mathbf{wMSO}$, denoted $\Phi \equiv \Xi$, is defined by $\text{free}(\Phi) = \text{free}(\Xi) = V$ and $\text{aggr}_{\text{sv}}(\llbracket \Phi \rrbracket_V(\bar{w})) = \llbracket \Xi \rrbracket_V(\bar{w})$ for all $\bar{w} \in \Sigma_V^+$.

Proposition 22 *Every formula Φ of $\mathbf{core-wMSO}(\Sigma, R)$ can be translated into an equivalent $\forall$ - and $\wedge$ -restricted formula $\tilde{\Phi}$ of $\mathbf{wMSO}'(\Sigma, R)$.*

Proof Since $\mathbf{step-wMSO}$ formulae are syntactically identical with step formulae of $\mathbf{wMSO}'$, we simply set $\tilde{\Psi} = \Psi$. For $\mathbf{core-wMSO}$ formulae Φ , the translation is performed inductively by

$$\begin{aligned} \phi ? \widetilde{\Phi_1} : \widetilde{\Phi_2} &= \phi ? \tilde{\Phi}_1 : \tilde{\Phi}_2 & \tilde{0} &= 0 \\ \widetilde{\Phi_1 + \Phi_2} &= \tilde{\Phi}_1 \vee \tilde{\Phi}_2 & \widetilde{\sum_x \Phi_1} &= \exists x \tilde{\Phi}_1 \\ \widetilde{\sum_x \Phi_1} &= \exists x \tilde{\Phi}_1 & \widetilde{\prod_x \Psi} &= \forall x \tilde{\Psi}. \end{aligned}$$

Notice that $0 \in R$ so that $\tilde{\Phi}$ is always a $\forall$ - and $\wedge$ -restricted formula of $\mathbf{wMSO}'(\Sigma, R)$. Let Φ in $\mathbf{core-wMSO}$ and let $\bar{w} \in \Sigma_V^+$ with $\text{free}(\Phi) \subseteq V$. If $\bar{w}$ is not valid then $\llbracket \Phi \rrbracket_V(\bar{w}) = \emptyset$ and $\llbracket \tilde{\Phi} \rrbracket_V(\bar{w}) = 0$. We conclude with $\text{aggr}_{\text{vm}}(\emptyset) = 0$. We assume now that $\bar{w} = (w, \sigma)$ is valid. We show by induction that $\text{aggr}_{\text{sv}}(\llbracket \Phi \rrbracket_V(w, \sigma)) = \llbracket \tilde{\Phi} \rrbracket_V(w, \sigma)$.

The case $\Phi = \phi ? \Phi_1 : \Phi_2$ is trivial. For 0 , $+$, $\sum_x$ and $\sum_X$, the result follows from the fact that the aggregation operator $\text{aggr}_{\text{vm}} = \sum \text{Val}$ is a monoid morphism from $(\mathbb{N}\langle R^* \rangle, \uplus, \emptyset)$ to $(S, +, 0)$, i.e. $\text{aggr}_{\text{vm}}(\emptyset) = 0$ and for all multisets $A, B \in \mathbb{N}\langle R^* \rangle$, we have

$$\text{aggr}_{\text{vm}}(A \uplus B) = \text{aggr}_{\text{vm}}(A) + \text{aggr}_{\text{vm}}(B).$$

Finally, assume that $\Phi = \prod_x \Psi$ and $\text{pos}(w) = \{1, \dots, n\}$. For every position $i \in \text{pos}(w)$, there exists a constant $r_i = \llbracket \tilde{\Psi} \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i]) \in S$ such that the abstract semantics is $\llbracket \Psi \rrbracket_{V \cup \{x\}}(w, \sigma[x \mapsto i]) = \{r_i\}$. We obtain

$$\begin{aligned} \text{aggr}_{\text{vm}}(\llbracket \prod_x \Psi \rrbracket_V(w, \sigma)) &= \sum \text{Val}(\{r_1 \dots r_n\}) \\ &= \text{Val}(r_1 \dots r_n) \\ &= \llbracket \forall x \tilde{\Psi} \rrbracket_V(w, \sigma). \end{aligned}$$

□

We now consider the reverse translation from $\mathbf{wMSO}'$ to $\mathbf{core-wMSO}$. We first define the hypothesis on the weight domain allowing us to obtain the result.

A subset R of a product valuation monoid $(S, +, 0, \text{Val}, \diamond, 1)$ is said to be regular if for every weight $r \in R$, there exists a sentence Φ_r of $\mathbf{core-wMSO}(\Sigma, S)$ such that

$$\text{aggr}_{\text{vm}}(\llbracket \Phi_r \rrbracket(w)) = r$$

for every $w \in \Sigma^+$.⁷ Notice that the formula Φ_r may use all possible constants in S , and not only the ones in R .

Example 23 In case of a valuation that is a λ -discounted sum, we may use the formula

$$\Phi_r = \prod_x \text{first}(x) ? r : 0$$

with $\text{first}(x)$ the $\mathbf{MSO}$ formula stating that x is the first position of the word. Indeed, Φ_r maps a word w of length n to $r + \lambda \cdot 0 + \lambda^2 \cdot 0 + \dots + \lambda^{n-1} \cdot 0 = r$. Another example is when the valuation takes the average of the weights. Then, the formula

$$\Phi_r = \prod_x r$$

is appropriate. Indeed, it maps w to $\frac{r+\dots+r}{n} = r$. As last example, in every semiring S , S is a regular subset using the formula

$$\Phi_r = \prod_x \text{first}(x) ? r : 1$$

and the fact that 1 is a unit for multiplication.

Proposition 24 *If the set R is regular in S , each $\forall$ - and $\wedge$ -restricted formula $\Xi \in \mathbf{wMSO}'(\Sigma, R)$ can be translated into an equivalent formula $\tilde{\Xi} \in \mathbf{core-wMSO}(\Sigma, S)$.*

Proof Using the regularity of R , we give a translation of $\forall$ - and $\wedge$ -restricted formulae of $\mathbf{wMSO}'(\Sigma, R)$ into $\mathbf{core-wMSO}(\Sigma, S)$:

$$\begin{aligned} \tilde{r} &= \Phi_r & \phi ? \widetilde{\Xi_1} : \widetilde{\Xi_2} &= \phi ? \tilde{\Xi}_1 : \tilde{\Xi}_2 \\ \widetilde{\Xi \vee \Xi'} &= \tilde{\Xi} + \tilde{\Xi'} & \widetilde{\forall x \Xi} &= \prod_x \tilde{\Xi} \\ \widetilde{\exists x \Xi} &= \sum_x \tilde{\Xi} & \widetilde{\exists X \Xi} &= \sum_X \tilde{\Xi} \end{aligned}$$

Notice again that a step formula in $\mathbf{wMSO}'$ is already a $\mathbf{step-wMSO}$ formula and does not require any translation. Using a proof by induction as in Proposition 22, it is immediate to check that Ξ and $\tilde{\Xi}$ are equivalent. □

⁷ In Droste and Meinecke (2012), the regularity is defined with respect to some weighted automata. Here, we prefer to use a purely logical definition. However, using Corollary 11-2., those two definitions are equivalent.

We will now translate the $\forall$ -restricted fragment of $\mathbf{wMSO}'$ into $\mathbf{core-wMSO}$ (no restriction on $\wedge$). To this end, we first define on weight domains the hypothesis **(DC)** which states that $\diamond$ distributes over $+$, i.e. $r \diamond (s + t) = r \diamond s + r \diamond t$ and $(s + t) \diamond r = s \diamond r + t \diamond r$ for all $r, s, t \in S$, and $\diamond$ is **Val**-commutative, i.e. $\mathbf{Val}(r_1 \dots r_n) \diamond \mathbf{Val}(s_1 \dots s_n) = \mathbf{Val}((r_1 \diamond s_1) \dots (r_n \diamond s_n))$ for all $r_1, \dots, r_n, s_1, \dots, s_n \in S$.

Proposition 25 *If the set R is regular in S and under hypothesis **(DC)**, every $\forall$ -restricted formula Ξ of $\mathbf{wMSO}'(\Sigma, R)$ can be translated into an equivalent formula $\tilde{\Xi}$ of $\mathbf{core-wMSO}(\Sigma, S)$.*

Proof We use the translation of Proposition 24, adding the following rule to translate conjunctions:

$$\widetilde{\Xi \wedge \Xi'} = \tilde{\Xi} \diamond \tilde{\Xi'}.$$

For all $\forall$ -restricted formula Ξ of $\mathbf{wMSO}'$, $\tilde{\Xi}$ is now a formula of $\diamond$ -**core-wMSO**, as introduced in Sect. 4.4. Checking that Ξ and $\tilde{\Xi}$ are equivalent is performed as in Proposition 24 for all constructs but the conjunction. For the conjunction, we have to show that

$$\begin{aligned} \text{aggr}_{\text{vm}} \llbracket \Phi_1 \diamond \Phi_2 \rrbracket(w, \sigma) \\ = \text{aggr}_{\text{vm}} \llbracket \Phi_1 \rrbracket(w, \sigma) \diamond \text{aggr}_{\text{vm}} \llbracket \Phi_2 \rrbracket(w, \sigma) \end{aligned}$$

i.e. that the semantics of $\diamond$ is compositional. This holds since, for all $n > 0$ and $A, B \in \mathbb{N}\langle S^n \rangle$, we have

$$\begin{aligned} \text{aggr}_{\text{vm}}(A \diamond B) \\ &= \sum \{\{\mathbf{Val}(\gamma) \mid \gamma \in A \diamond B\}\} \\ &= \sum \{\{\mathbf{Val}(\alpha \diamond \beta) \mid \alpha \in A, \beta \in B\}\} \\ &= \sum \{\{\mathbf{Val}(\alpha) \diamond \mathbf{Val}(\beta) \mid \alpha \in A, \beta \in B\}\} \\ &= (\sum \{\{\mathbf{Val}(\alpha) \mid \alpha \in A\}\}) \diamond (\sum \{\{\mathbf{Val}(\beta) \mid \beta \in B\}\}) \\ &= \text{aggr}_{\text{vm}}(A) \diamond \text{aggr}_{\text{vm}}(B). \end{aligned}$$

By Proposition 16, we may finally translate the $\diamond$ -**core-wMSO** formula $\tilde{\Xi}$ into an equivalent **core-wMSO** formula. $\square$

Finally using Corollary 10, the previous propositions and Remark 21 linking fragments of $\mathbf{wMSO}$ and $\mathbf{wMSO}'$, we obtain

Theorem 26 *Suppose that S is regular. The following formalisms are expressively equivalent:*

- S -weighted automata over alphabet Σ ;
- $\mathbf{core-wMSO}(\Sigma, S)$;
- $\forall$ - and $\wedge$ -restricted $\mathbf{wMSO}'(\Sigma, S)$;
- $\forall$ -restricted $\mathbf{wMSO}'(\Sigma, S)$ *if (DC);*
- $\forall$ - and $\wedge$ -restricted $\mathbf{wMSO}(\Sigma, S)$ *if (01);*

- $\forall$ -restricted $\mathbf{wMSO}(\Sigma, S)$ *if (DC) and (01).*

Notice that the sets of constants in logics and automata are always taken to be S in order to simplify the statements. For the same reason, the regularity condition is on the full set of weights S . Previous propositions give more precise results on how the set of constants changes along the translations.

5.2 wMSO over valuation structures

The syntax of the logic $\mathbf{wMSO}$ over valuation structures (Droste and Perevoshchikov 2013) is the same as for valuation monoids defined before. The difference comes only from the semantics. Consider a valuation structure $(U, \mathbf{Val}, S, F)$. As previously, a binary operator $\diamond: U^2 \rightarrow U$, as well as a constant 1 in R , is necessary to define the semantics of conjunction and of the Boolean formulae. Such a tuple $(U, \mathbf{Val}, S, F, \diamond, 1)$ is called a product valuation structure. Notice that, contrary to product valuation monoids, the element 1 lies in U and not in S .

The easiest way to define the semantics in that case is to rely on the case of product valuation monoids. Indeed, notice that $(\mathbb{N}\langle U \rangle, \uplus, \emptyset, \mathbf{Val}, \diamond, \{1\})$ is a product valuation monoid. The diamond operator is lifted from the valuation structure to multisets pointwisely. The valuation of a sequence $(A_i)_{1 \leq i \leq n}$ of multisets is the multiset $\{\{\mathbf{Val}(a_1 \dots a_n) \mid a_i \in A_i\}\}$. Hence, this allows us to define the semantics of formulae Ξ of $\mathbf{wMSO}$ in two steps. First, a semantics $\langle \Xi \rangle_V(\bar{w})$ in the product valuation monoid $\mathbb{N}\langle U \rangle$ as defined in Table 4. Then, the final interpretation $F(\langle \Xi \rangle_V(\bar{w}))$ given by F . Notice the difference between the semi-interpreted semantics $\langle \cdot \rangle_V(\bar{w})$ and our abstract semantics $\llbracket \cdot \rrbracket_V(\bar{w})$ in the case of universal quantifications.

Theorem 26 also holds for product valuation structures with the hypotheses simplified as follows:

- the regularity condition on $\mathbb{N}\langle U \rangle$ is fulfilled when for every constant $r \in U$, there exists a sentence Φ_r of $\mathbf{core-wMSO}(\Sigma, U)$ such that $\llbracket \Phi_r \rrbracket(w) = \{r\}$ for all $w \in \Sigma^+$;
- hypothesis **(01)** is fulfilled when $1 \diamond r = r \diamond 1 = r$ for all $r \in U$, and $\mathbf{Val}(1 \dots 1) = 1$. In particular, conditions on the zero element $\emptyset \in \mathbb{N}\langle U \rangle$ are trivially verified because of the pointwise definition of $\diamond$ and $\mathbf{Val}$ over multisets;
- hypothesis **(DC)** is fulfilled when $\mathbf{Val}(r_1 \dots r_n) \diamond \mathbf{Val}(s_1 \dots s_n) = \mathbf{Val}((r_1 \diamond s_1) \dots (r_n \diamond s_n))$ for all $r_1, \dots, r_n, s_1, \dots, s_n \in U$. In particular, $\diamond$ always distributes over $\uplus$ in $\mathbb{N}\langle U \rangle$.

Indeed, Theorem 26 for product valuation structures is obtained by first applying Theorem 26 to the product valuation monoid $\mathbb{N}\langle U \rangle$ and by then applying the evaluator function F .

6 Extensions to ranked and unranked trees

In this section, we show how to extend the equivalence between weighted automata and **core-wMSO** to other structures, namely ranked and unranked trees. We will primarily use a semantics in multisets of weight trees (instead of weight sequences). Then, we may apply an aggregation operator to recover a more concrete semantics. This approach allows us to infer results for semirings (Droste 2006; Droste and Vogler 2011) and also for tree valuation monoids (Droste et al. 2011).

There are two main ingredients allowing us to prove the equivalence between **core-wMSO** and weighted automata. First, in the Boolean case, we should have an equivalence between unambiguous (or deterministic) automata and **MSO** logic. This equivalence is known for many structures such as words (Büchi 1960; Elgot 1961; Trakhtenbrot 1961), ranked trees (Thatcher and Wright 1968), unranked trees (Comon-Lundh et al. 2008), etc. Second, the computation of the weight of a run ρ of an automaton, and the evaluation of a product formula $\prod_x \Psi$ should be based on the same mechanism. For words and valuation monoids (or valuation structures), it is the valuation of a sequence of weights. This is why we used an abstract semantics in the semiring of multisets of weight sequences. For trees and tree valuation monoids, the valuation takes a tree of weights as input and returns a value in the monoid. Hence, we use multisets of weight trees as abstract semantics. Note that, multisets of weight trees form a monoid but not a semiring.⁸

6.1 Ranked and unranked trees

Let $\mathbb{P} = \{1, 2, \dots\}$ be the set of positive natural numbers and $\mathbb{P}^*$ be the set of finite words over $\mathbb{P}$. A tree domain $\mathcal{B}$ is a finite, non-empty subset of $\mathbb{P}^*$ such that if $u \cdot i \in \mathcal{B}$ with $u \in \mathbb{P}^*$ and $i \in \mathbb{P}$, then we have $u, u \cdot 1, \dots, u \cdot (i - 1) \in \mathcal{B}$. An unranked tree over a set Σ of labels (Σ -tree) is a partial mapping $t: \mathbb{P}^* \rightarrow \Sigma$ with a $\text{dom}(t) \subseteq \mathbb{P}^*$ being a tree domain. The arity of a node $u \in \text{dom}(t)$ is $\text{ar}(u) = \max\{i \in \mathbb{P} \mid u \cdot i \in \text{dom}(t)\}$, with $\max \emptyset = 0$. We denote by UT_Σ the set of unranked Σ -trees.

A ranked alphabet is a pair (Σ, rk) with Σ a finite alphabet and $\text{rk}: \Sigma \rightarrow \mathbb{N}$ giving the rank of each symbol in Σ . For each rank m , we denote by $\Sigma^{(m)}$ the set of symbol $a \in \Sigma$ with $\text{rk}(a) = m$. A ranked tree over (Σ, rk) is a Σ -tree t such that for each $u \in \text{dom}(t)$ we have $\text{ar}(u) = \text{rk}(t(u))$.

An (unranked) tree automaton over Σ is a tuple $\mathcal{A} = (Q, \Delta, F)$ with Q a non-empty finite set of states, $F \subseteq Q$ the set of accepting states, and $\Delta \subseteq \text{Reg}(Q^*) \times \Sigma \times Q$ a finite set of transitions, where $\text{Reg}(Q^*)$ denotes the set of regular languages over alphabet Q . Here we may (finitely)

⁸ The product in the semiring of weight sequences is based on the concatenation of sequences. There is no natural counterpart for trees.

represent regular languages by finite-state automata or regular expressions over the alphabet Q .

A run of $\mathcal{A}$ over a Σ -tree t is a Δ -tree ρ with $\text{dom}(\rho) = \text{dom}(t)$ and such that for all $u \in \text{dom}(t)$ we have $\rho(u) = (L(u), t(u), q(u)) \in \Delta$ and $q(u \cdot 1) \dots q(u \cdot \text{ar}(u)) \in L(u)$. The run is accepting if $q(\varepsilon) \in F$.⁹

The language $L(\mathcal{A})$ accepted by the tree automaton $\mathcal{A}$ is the set of Σ -trees on which there exists at least one accepting run.

Remark 27 In case of a ranked alphabet, we recover the classical ranked tree automata with $\Delta \subseteq \bigcup_m Q^m \times \Sigma^{(m)} \times Q$.

A tree automaton $\mathcal{A}$ is said to be deterministic if for all distinct transitions (L, a, q) and (L', a, q') in Δ , we have $L \cap L' = \emptyset$. Equivalently, for every word $\pi \in Q^*$ and $a \in \Sigma$, there is at most one transition $(L, a, q) \in \Delta$ such that $\pi \in L$. Hence, every tree t admits at most one run. It is well known that tree automata can be determined (see, e.g. (Comon-Lundh et al. 2008)).

6.2 Weighted automata over trees

An R -weighted (unranked) tree automaton over Σ is a tuple $\mathcal{A} = (Q, \Delta, \text{wgt}, F)$ with (Q, Δ, F) a tree automaton and $\text{wgt}: \Delta \rightarrow R$ associating a weight to every transition.

The weight tree arising from a run ρ of $\mathcal{A}$ over a Σ -tree t is the R -tree $\text{wgt} \circ \rho$ mapping each $u \in \text{dom}(t)$ to $\text{wgt}(\rho(u)) \in R$. The abstract semantics of an R -weighted tree automaton $\mathcal{A}$ is a multiset of weight trees. For all trees $t \in \text{UT}_\Sigma$, we define

$$\llbracket \mathcal{A} \rrbracket(t) = \{\{\text{wgt} \circ \rho \mid \rho \text{ is an accepting run of } \mathcal{A} \text{ on } t\}\}.$$

Hence, our abstract semantics lives in the commutative monoid $\mathbb{N}\langle \text{UT}_R \rangle$ of multisets of R -trees. Then, we may use an aggregation operator $\text{aggr}: \mathbb{N}\langle \text{UT}_R \rangle \rightarrow S$ to obtain a concrete semantics in a possibly different weight structure S :

$$\llbracket \mathcal{A} \rrbracket(t) = \text{aggr}(\llbracket \mathcal{A} \rrbracket(t)).$$

Example 28 (Weighted automata over semirings) In the classical setting, the set R of weights is a subset of a semiring $(S, +, \times, 0, 1)$. The value of a run ρ of $\mathcal{A}$ over a Σ -tree t is the product of the weights in the R -tree $\text{wgt} \circ \rho$. Since the semiring is not necessarily commutative, we have to specify the order in which this product is computed. Classically, we choose the postfix order. Formally, given an R -tree v , the

⁹ Equivalently, we may assume that transitions are disjoint: if $(L, a, q), (L', a, q') \in \Delta$ then $L = L'$ or $L \cap L' = \emptyset$. In this case, a run can be defined as a simpler Q -tree ρ with $\text{dom}(\rho) = \text{dom}(t)$ and such that for all $u \in \text{dom}(t)$ there is a (unique) transition $(L, t(u), \rho(u)) \in \Delta$ with $\rho(u \cdot 1) \dots \rho(u \cdot \text{ar}(u)) \in L$.

product $\prod(v) = \text{Prod}(v, \varepsilon)$ is computed bottom-up: for all $u \in \text{dom}(v)$, we set

$$\text{Prod}(v, u) = \text{Prod}(v, u \cdot 1) \times \cdots \times \text{Prod}(v, u \cdot \text{ar}(u)) \\ \times v(u).$$

Note that, if u is a leaf then $\text{Prod}(v, u) = v(u)$. As for words, the mapping $\prod: \text{UT}_R \rightarrow S$ can be lifted to a mapping $\prod: \mathbb{N}(\text{UT}_R) \rightarrow \mathbb{N}(S)$.

Then, the semantics is defined as always by summing the values of the accepting runs: $\llbracket \mathcal{A} \rrbracket(t) = \sum_{\rho} \prod(\text{wgt}(\rho))$ where the sum ranges over accepting runs ρ of $\mathcal{A}$ over the Σ -tree t . Therefore, the classical case of semirings is obtained from the abstract semantics with the aggregation operator

$$\text{aggr}_{\text{sr}}(A) = \sum \prod(A) = \sum_{v \in A} \prod(v).$$

In the case of a ranked alphabet, we recover the definition of Droste (2006) of weighted tree automata. The comparison with the weighted unranked tree automata of Droste and Vogler (2011) is not as easy, at least over non-commutative semirings. We believe that over commutative semirings, our model is equivalent to the weighted unranked tree automata of Droste and Vogler (2011). The situation is different over non-commutative semirings. Our definition is best motivated by considering words as special cases of trees. There are two ways to inject words in unranked trees, as shown in Fig. 2: either in a horizontal way (a root with children representing the word from left to right), or a vertical way (unary nodes followed by a leaf, the word being read from bottom to top). With some easy encodings, we may see that our model of weighted unranked tree automata is a conservative extension of weighted word automata, both for the horizontal and the vertical injections of words. Moreover, our approach allows us to obtain the equivalence between automata and logic for arbitrary semirings (even non-commutative ones), as stated in Theorem 30.

In contrast, the model of Droste and Vogler (2011) is not a conservative extension of weighted word automata for the horizontal injection. This is witnessed by an example given in Theorem 6.10 (Droste and Vogler 2011) that we now recall. In the (non-commutative) semiring $(\mathfrak{P}(\{p, q\}^*),$

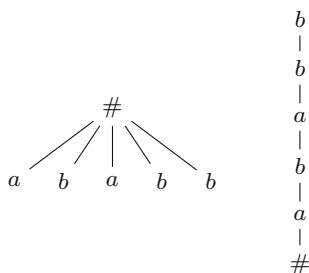


Fig. 2 Horizontal and vertical injection of the word $ababb$ into trees

$\cup, \cdot, \emptyset, \{\varepsilon\}$), with two distinct letters p and q , we consider $f: \text{UT}_\Sigma \rightarrow \mathfrak{P}(\{p, q\}^*)$ the tree series mapping every tree t composed of a root directly followed by n children ($n \in \mathbb{N}$) to the language $\{p^n q^n\}$, and every other tree to $\emptyset$. The model of weighted unranked tree automata we have chosen cannot recognise this tree series. However, the model of automata described in Droste and Vogler (2011) is able to recognise this tree series. The main difference between the two models, which explains this discrepancy, is the way weights are assigned during the computation of the automaton. While we have decided to assign weights to transitions of the unranked tree automaton, keeping a Boolean regular (hedge) language to determine whether a transition is enabled, Droste and Vogler decided instead to use a weighted (hedge) automaton when reading the sequence of states of the children. Then, to each position in the tree domain is associated the weight of the (hedge) automaton reading the sequence of states of the children. The semantics over a tree is given by a depth-first left-to-right product of those weights (first the weight of the children from left to right, and then the weight of the parent).

Example 29 [Tree valuation monoids] As for words, extensions of weighted automata to more general weight domains have been considered. Following (Droste et al. 2011), a tree valuation monoid is a tuple $(S, +, 0, \text{Val})$ where $(S, +, 0)$ is a commutative monoid and $\text{Val}: \text{UT}_S \rightarrow S$ is a valuation function from S -trees to S . The value of a run ρ is now computed by applying this valuation function to the R -tree $\text{wgt} \circ \rho$. The final semantics is obtained as above by summing the values of accepting runs. Therefore, the semantics in tree valuation monoids is obtained from the abstract semantics with the aggregation operator

$$\text{aggr}_{\text{vm}}(A) = \sum \text{Val}(A) = \sum_{v \in A} \text{Val}(v).$$

For instance, when $(S, +, \times, 0, 1)$ is a semiring, we obtain a tree valuation monoid with the postfix product $\prod$ defined in Example 28. We refer to (Droste et al. 2011) for other examples of weighted ranked tree automata, including the interesting case of multi-operator monoids which is also studied in Fülöp (2012). A further extension for unranked trees has recently been considered in Droste et al. (2015).

Further extensions like tree valuation structures—where the sum in tree valuation monoids is replaced by a more general operator F as for words—are also possible, though not considered in the literature so far. Our results will apply in this context as well.

6.3 core-wMSO over trees

There are only very few changes needed to lift the logic core-wMSO defined in Sect. 4 to ranked or unranked trees.

On one hand, we need to change the atomic predicates of the Boolean fragment **MSO** to the corresponding ones over trees. For unranked trees, we use $\text{child}(x, y)$ which says that under the current valuation σ we have $\sigma(y) = \sigma(x) \cdot i$ for some $i \in \mathbb{P}$, and $\text{nextsibling}(x, y)$ for $\sigma(x) = u \cdot i$ and $\sigma(y) = u \cdot (i + 1)$ for some $u \in \mathbb{P}^*$ and $i \in \mathbb{P}$. For ranked trees, we only use $\text{child}_i(x, y)$ which means that y is the i th child of x ($\sigma(y) = \sigma(x) \cdot i$). Thanks to (Thatcher and Wright 1968) for ranked trees and (Comon-Lundh et al. 2008) for unranked trees, we know that **MSO** has the same expressive power as deterministic (bottom-up) tree automata.

On the other hand, we need to give the semantics of **core-wMSO** formulæ in the setting of trees. Once again, we will use the classical encoding of a pair (t, σ) consisting of a Σ -tree t and a valuation σ from a set V of variables to the domain of t , as a Σ_V -tree $\bar{t}$. The notion of validity of such a Σ_V -tree $\bar{t}$ is defined as for words. The semantics of formulæ of **step-wMSO** maps Σ_V -trees to $\mathbb{N}\langle R \rangle$, and is defined as in Table 2 using $t, \sigma \models \phi$ for **MSO** formulæ ϕ . For **core-wMSO**, in case of a valid Σ_V -tree $\bar{t} = (t, \sigma)$, the semantics is defined as in Table 3 except for the operator $\prod_x$ for which we let

$$\|\prod_x \Psi\|_V(t, \sigma) = \{\{t'\}\}$$

where t' is the R -tree with the same domain as t such that for all $u \in \text{dom}(t)$ we have $\|\Psi\|_V(t, \sigma[x \mapsto u]) = \{\{t'(u)\}\}$.

Then, we obtain the following result.

Theorem 30 *For each R -weighted tree automaton $\mathcal{A}$ over alphabet Σ , we can effectively construct a sentence $\Phi_{\mathcal{A}}$ in **core-wMSO**(Σ, R), such that $\|\mathcal{A}\|(t) = \|\Phi_{\mathcal{A}}\|(t)$ for all Σ -trees t .*

*For each sentence Φ in **core-wMSO**(Σ, R), we can effectively construct an R -weighted tree automaton $\mathcal{A}_{\Phi}$ over Σ such that $\|\Phi\|(t) = \|\mathcal{A}_{\Phi}\|(t)$ for all Σ -trees t .*

Proof The proof of Theorem 9 given in Sect. 4.5 may easily be adapted to the case of trees. In particular, Lemma 18 can be proved mutatis mutandis.

Then, in the translation from automata to the logic, only the formula $\text{run}(\bar{X})$ has to be modified to check that $\bar{X}$ encodes a run of $\mathcal{A}$ over a given tree t . For that formula, we still have to check that $\bar{X}$ is a partition of the positions of t that the root is labelled with a transition $\delta \in \text{Reg}(Q^*) \times \Sigma \times F$, and that for each node u of t labelled with transition $\delta = (L, a, q)$, the sequence $q_1 \dots q_{\text{ar}(u)} \in L$ where (L_i, a_i, q_i) is the transition associated with the child $u \cdot i$ of u . This last condition is checked with the help of an **MSO** formula over words equivalent to the regular language L , replacing the linear order $\leq$ over words with the reflexive and transitive

closure of the sibling relation $\text{nextsibling}(x, y)$.¹⁰ Notice also that the subformula $\prod_x \text{weight}(x, \bar{X})$ in $\Phi_{\mathcal{A}}$ correctly computes $\{\{\text{wgt} \circ \rho\}\}$ where ρ is the run encoded by $\bar{X}$.

For the translation from logic to automata, there are also some minor adaptations to tree automata. For the projection, let us explain how to construct an automaton $\mathcal{A}_{\Phi}$ from $\mathcal{A}_{\Phi_1}$ where $\Phi = \sum_X \Phi_1$. As for the word case, we transfer the alphabet component for X of $\mathcal{A}_{\Phi_1}$ in the state of $\mathcal{A}_{\Phi}$. We denote by π_1 the projection from $Q \times \{0, 1\}$ to Q . Then, if $\mathcal{A}_{\Phi_1} = (Q, \Delta, \text{wgt}, F)$ then $\mathcal{A}_{\Phi} = (Q \times \{0, 1\}, \Delta', \text{wgt}', F \times \{0, 1\})$ with

$$(\pi_1^{-1}(L), a, (q, j)) \in \Delta' \text{ iff } (L, (a, j), q) \in \Delta$$

with weights inherited by

$$\text{wgt}'(\pi_1^{-1}(L), a, (q, j)) = \text{wgt}(L, (a, j), q).$$

A similar modification is made for $\sum_x$ in order to check during the computation that exactly one 1 appears in the x component. We use the second projection π_2 from $Q \times \{0, 1\}$ to $\{0, 1\}$. Then, transitions are given by

$$\begin{aligned} (\pi_1^{-1}(L) \cap \pi_2^{-1}(0^*), a, (q, j)) &\in \Delta' \text{ if } (L, (a, j), q) \in \Delta \\ (\pi_1^{-1}(L) \cap \pi_2^{-1}(0^*10^*), a, (q, 1)) &\in \Delta' \text{ if } (L, (a, 0), q) \in \Delta \end{aligned}$$

and weights are inherited as before.

Next, for the if-then-else operator, we use the translation of a Boolean **MSO** formula into a deterministic tree automaton (Comon-Lundh et al. 2008). The cartesian product of a deterministic tree automaton and a weighted tree automaton can be adapted easily.

Finally, the construction for $\Phi = \prod_x \Psi$ can be easily adapted. Again, we assume that Ψ is a formula in **set-step-wMSO**. The tree automaton $\mathcal{A}_{\Phi}$ has a single state q which is accepting. For every letter $(a, \tau) \in \Sigma_{\text{free}(\Phi)}$, there is a transition $(\{q\}^*, (a, \tau), q)$ with weight $\Psi(\tau)$. The automaton $\mathcal{A}_{\Phi}$ is deterministic and complete. It has a single run ρ on each $\Sigma_{\text{free}(\Phi)}$ -tree $\bar{t} = (t, \sigma)$ and for all nodes $u \in \text{dom}(t)$ we have $\|\Psi\|(t, \sigma[x \mapsto u]) = \{\{\text{wgt}(\rho(u))\}\}$. We deduce that $\|\mathcal{A}_{\Phi}\|(\bar{t}) = \{\{\text{wgt} \circ \rho\}\} = \|\Phi\|(\bar{t})$. $\square$

7 Conclusion

We proved the meta-theorem relating weighted automata and **core-wMSO** at the level of multisets of weight structures for words and trees. However, the definitions and techniques developed in this article can easily be adapted for other structures like nested words, Mazurkiewicz traces, etc. The logical

¹⁰ An even simpler formula can be obtained in the simpler setting of ranked trees.

equivalence between restricted wMSO and core-wMSO at the concrete level is established for words in Sect. 5. An analogous result can be obtained for trees with a similar logical reasoning. In particular, this allows for an extension to trees of the valuation structures of (Droste and Perevoshchikov 2013).

In this article, our meta-theorem is only stated and proved for finite structures. At the level of the concrete semantics, equivalences between weighted automata and weighted logics have been extended to infinite structures, such as words or trees over semirings (Droste and Gastin 2007), valuation monoids (Droste and Meinecke 2012) or valuation structures (Droste and Perevoshchikov 2013). An extension of our meta-theorem to infinite structures capturing these results is a natural open problem. Finite multisets of weight structures are not adequate anymore since automata may exhibit infinitely many runs on a given input structure. The abstract semantics should ideally distinguish between countably many runs or uncountably many runs.

Compliance with ethical standards

Conflict of interest The authors declare that they have no conflict of interest.

References

- Albert J, Kari J (2009) Digital image compression. In: Kuich W, Vogler H, Droste M (eds) *Handbook of Weighted Automata*, EATCS Monographs in Theoretical Computer Science, chap. 5. Springer, pp. 445–472
- Bollig B, Gastin P (2009) Weighted versus probabilistic logics. In: *Proceedings of the 13th international conference on developments in language theory (DLT'09)*, Lecture Notes in Computer Science, vol. 5583, Springer, pp. 18–38
- Büchi JR (1960) Weak second-order arithmetic and finite automata. *Zeitschrift für Mathematische Logik und Grundlagen der Mathematik* 6:66–92
- Černý P, Chatterjee K, Henzinger TA, Radhakrishna A, Singh R (2011) Quantitative synthesis for concurrent programs. In: *Proceedings of the 23rd international conference on computer aided verification (CAV'11)*, Lecture Notes in Computer Science, vol 6806, Springer, pp. 243–259
- Comon-Lundh H, Dauchet M, Gilleron R, Jacquemard F, Lugiez D, Tison S, Tommasi M (2008) Tree automata techniques and applications. <http://tata.gforge.inria.fr/>
- Droste M, Gastin P (2007) Weighted automata and weighted logics. *Theor Comput Sci* 380(1–2):69–86
- Droste M, Götze D, Märcker S, Meinecke I (2011) Weighted tree automata over valuation monoids and their characterization by weighted logics. In: *Algebraic Foundations in Computer Science, Lecture Notes in Computer Science*, vol 7020, Springer, pp. 30–55
- Droste M, Heusel D, Vogler H (2015) Weighted unranked tree automata over tree valuation monoids and their characterization by weighted logics. In: *Proceedings of the 6th international conference on algebraic informatics (CAI'15)*, Lecture Notes in Computer Science, vol 9270, Springer, pp. 90–102
- Droste M, Kuich W (2009) Semirings and formal power series. In: Droste M, Kuich W, Vogler H (eds) *Handbook of weighted automata*, EATCS monographs in theoretical computer science, chap. 1, Springer, pp. 3–27
- Droste M, Meinecke I (2012) Weighted automata and weighted MSO logics for average and long-time behaviors. *Inf Comput* 220–221:44–59
- Droste M, Perevoshchikov V (2013) Multi-weighted automata and MSO logic. In: *Proceedings of the 8th international computer science symposium in Russia (CSR'13)*, Lecture Notes in Computer Science, vol. 7913, Springer, pp 418–430
- Droste M, Perevoshchikov V (2014) A Nivat theorem for weighted timed automata and weighted relative distance logic. In: *Proceedings of the 41st international colloquium on automata, languages and programming (ICALP'14)*, Lecture Notes in Computer Science, vol. 8573, Springer, pp. 171–182
- Droste M, Vogler H (2006) Weighted tree automata and weighted logics. *Theor Comput Sci* 366(3):228–247
- Droste M, Vogler H (2011) Weighted logics for unranked tree automata. *Theory Comput Syst* 48:23–47
- Elgot CC (1961) Decision problems of finite automata design and related arithmetics. *Trans Am Math Soc* 98:21–52
- Fülöp Z, Stüber T, Vogler H (2012) A Büchi-like theorem for weighted tree automata over multioperator monoids. *Theory Comput Syst* 50:241–278
- Knight K, May J (2009) Applications of weighted automata in natural language processing. In: Droste M, Kuich W, Vogler H (eds) *Handbook of weighted automata*, EATCS monographs in theoretical computer science, chap. 14, Springer, pp 555–579
- Kuich W, Salomaa A (1985) *Semirings. Automata and Languages*. EATCS Monographs in Theoretical Computer Science. Springer-Verlag
- Perevoshchikov V (2015) Multi-weighted automata models and quantitative logics. Ph.D. thesis, Universität Leipzig
- Schützenberger MP (1961) On the definition of a family of automata. *Inf Control* 4:245–270
- Thatcher JW, Wright JB (1968) Generalized finite automata theory with an application to a decision problem of second-order logic. *Math Syst Theory* 2(1):57–81
- Trakhtenbrot BA (1961) Finite automata and logic of monadic predicates. *Doklady Akademii Nauk SSSR* 149:326–329